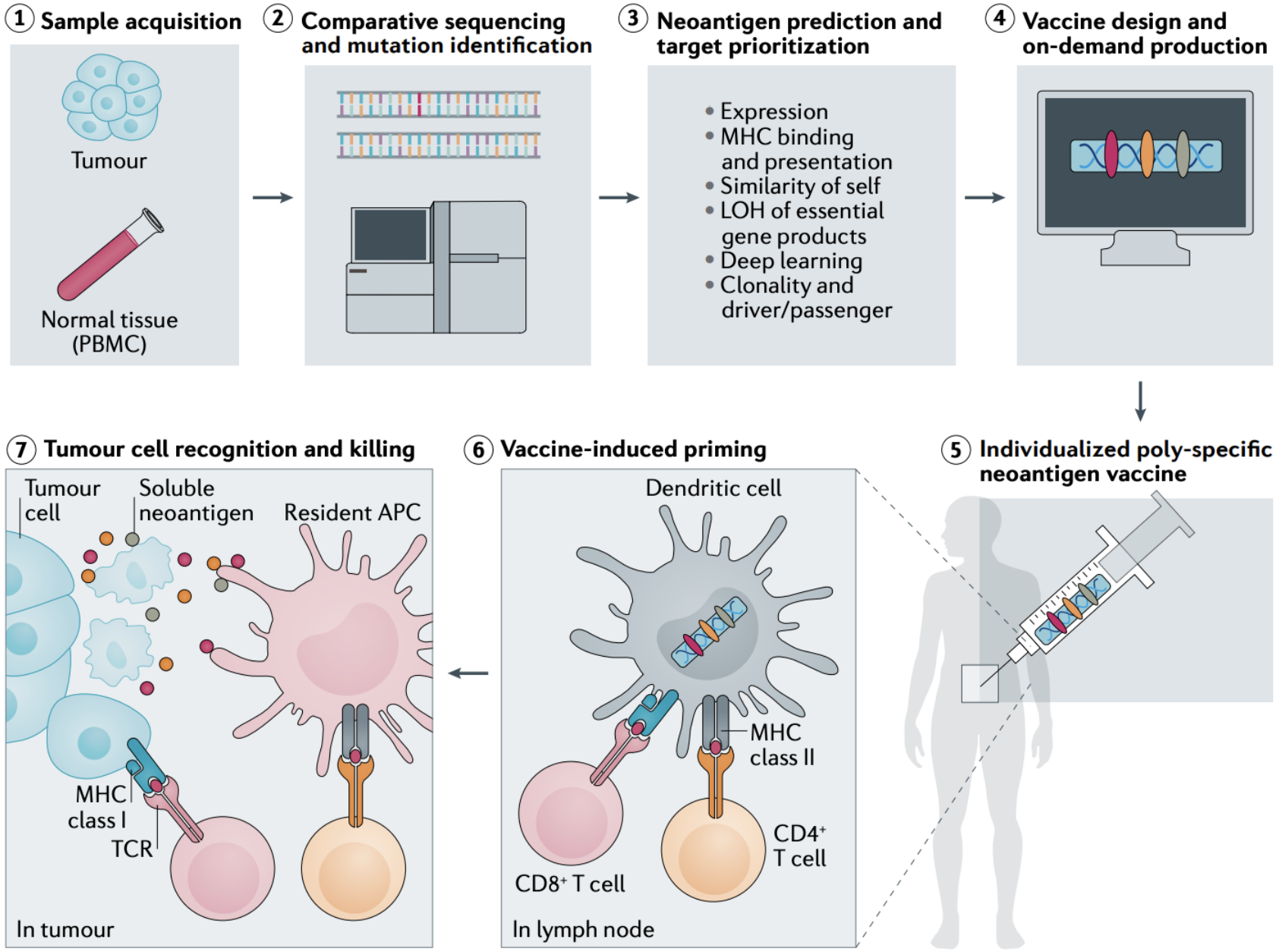
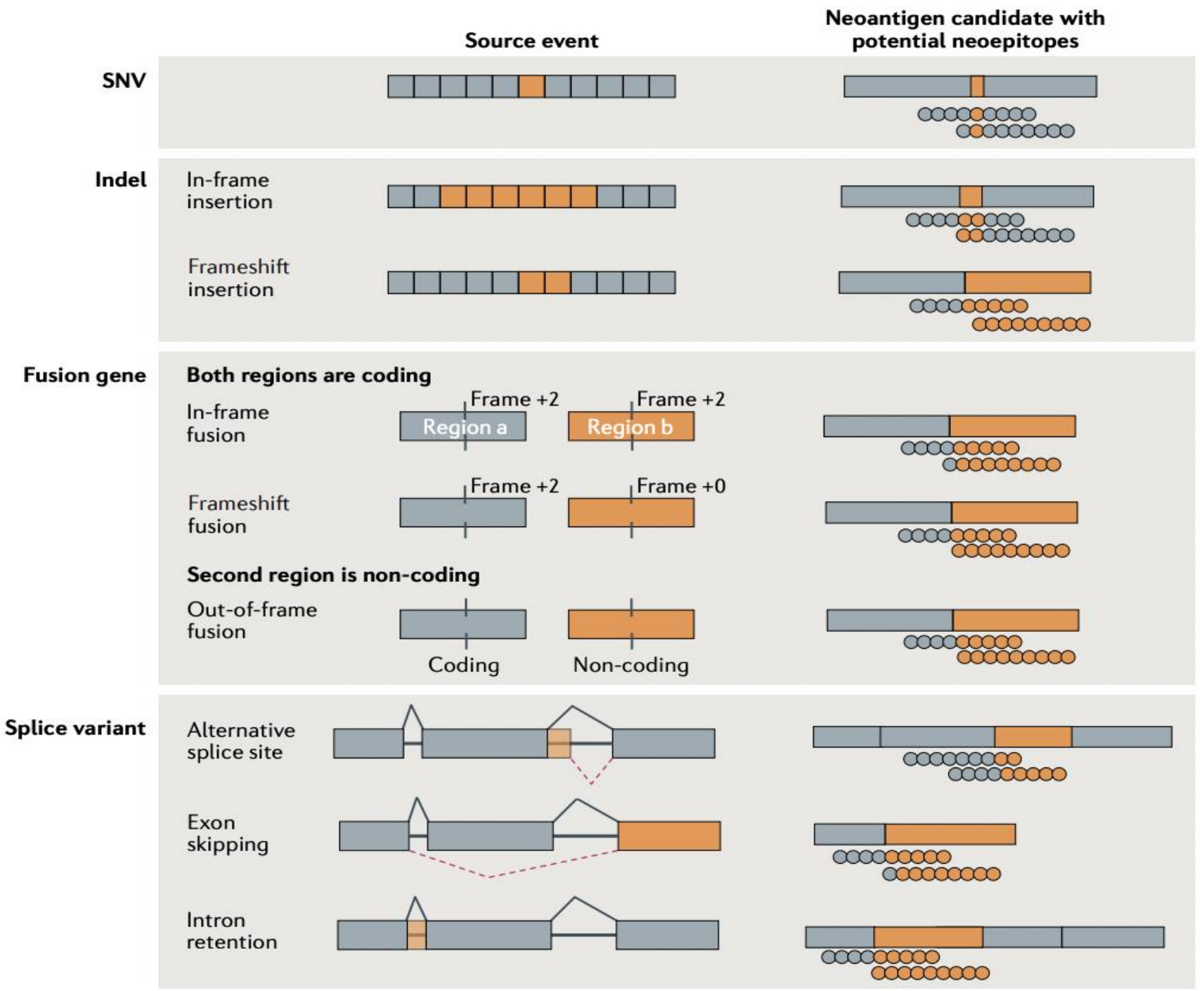
Peptide-MHC-TCR Binding

Patrick


1 Sample acquisition
Tumour
Normal tissue (PBMC)
2 Comparative sequencing and mutation identification
3 Neoantigen prediction and target prioritization
• Expression
• MHC binding and presentation
• Similarity of self
• LOH of essential gene products
• Deep learning
• Clonality and driver/passenger
4 Vaccine design and on-demand production
5 Individualized poly-specific neoantigen vaccine
6 Vaccine-induced priming
Dendritic cell
MHC class II
CD4+ T cell
CD8+ T cell
In lymph node
7 Tumour cell recognition and killing
Tumour cell
Soluble neoantigen
Resident APC
MHC class I
TCR
In tumour

Identification of Tumor-specific Mutations

Exome sequence of normal Vs. Cancer tissues -> somatic mutations



Selection of Neoantigens

Likelihood to stimulate T-cell response

- Peptide and MHC binding prediction
- Quantification of mutated transcript expression
- Clonality of mutations

Name	Function	Input	Comment
MHC-I binding	MHC-I binding prediction	Neoantigen candidate sequence, MHC-I alleles	Several tools (e.g. NetMHCpan)[136]
MHC-II binding	MHC-II binding prediction	Neoantigen candidate sequence, MHC-II alleles	Several tools (e.g. NetMHCIIpan)[251]
MixMHCpred score[138,139]	Prediction of cell surface presentation of MHC-I epitopes	Neoepitope candidate sequence, MHC-I alleles	Trained on eluted ligands
MixMHC2pred score[252]	Prediction of cell surface presentation of MHC-II epitopes	Neoepitope candidate sequence, MHC-II alleles	Trained on eluted ligands
Transcript expression[15,32]	Transcript expression in FPKM, RPKM or TPM	RNA-seq data	Several tools (e.g. kallisto)[132]
Clonality[110]	The fraction of tumour clones in which a neoantigen is present	Number of reads that cover WT and mutated allele	Several tools (e.g. PyClone)[156]
Differential agretopicity index (DAI)[109,225]	Difference in MHC-I binding affinities between mutated and WT peptide	MHC-I binding affinity of neoepitope candidate and corresponding WT	NA
Self-similarity[150]	Sequence similarity between mutated and WT peptide	Neoepitope candidate and corresponding WT sequence	Relevant for neoepitopes with similar MHC binding to WT peptide
Pathogen similarity[67]	Similarity to known pathogen epitopes	Neoepitope candidate sequence	BLAST search against pathogen-derived epitopes IEDB
IEDB immunogenicity[253]	Summed up position-weighted enrichment of residues in immunogenic peptide sequences	Neoepitope candidate sequence	Available as a webserver
Neoantigen dissimilarity[151]	Dissimilarity of epitope sequence to self-proteome	Neoepitope candidate sequence	BLAST against WT proteome
PHBR-I[146]	Mutation presentation by multiple patient MHC-I alleles	Best MHC binding score for each MHC-I allele	Considers multiple neoepitope candidates per mutation
PHBR-II[147]	Mutation presentation by multiple patient MHC-II alleles	Best MHC binding score for each MHC-II allele	Considers multiple neoepitope candidates per mutation
Generator rate[254]	Mutation presentation by multiple neoepitopes	MHC-I binding affinities	Considers multiple neoepitope candidates per mutation
Recognition potential[67,112]	Combinatorial score considering pathogen similarity and differential MHC binding of mutated and WT peptide	Neoepitope candidate sequence, MHC-I binding affinities	Considers multiple neoepitope characteristics
Vaxrank[255,256]	Combinatorial score considering the presentation of a mutation by multiple epitopes and variant allele expression	Variant allele expression, MHC-I binding affinities of neoepitope candidates related to a mutation	Considers multiple neoepitope candidates per mutation
Priority score[257]	Combinatorial score considering MHC binding and RNA expression	MHC-I binding affinity of neoepitope candidate and WT, transcript expression, variant allele frequency	Considers multiple neoepitope characteristics
T cell predictor[258]	Random forest model considering several epitope features such as expression, hydrophobicity	Gene name, substitution, neoepitope candidate sequence	Considers multiple neoepitope characteristics
neoag[259]	Gradient boosting model considering epitope sequence intrinsic features	Neoepitope candidate sequence, corresponding WT sequence, variant position, MHC-I binding	Trained on an immunogenicity data set from mouse tumour models

MHCAttnNet: predicting MHC-peptide bindings for MHC alleles classes I and II using an attention-based deep neural model

Step 1: Obtain word embeddings.

- Model: word2vec skip-gram
- Input: Protein database "uniprot_sprot" FASTA file
- Output: numerical embedding for k-mers
  e.g., 1-mer ['unnamed', 'padding', 'L', 'A',…, 'C', 'W', 'X']--> [23, 100], in [query, dimension]
  3-mer ['unnamed', 'padding', 'AAA', 'ALA', …, 'XRH']-->[365, 100],

Step 2: Select MHC Allele, Peptides and define binding scores

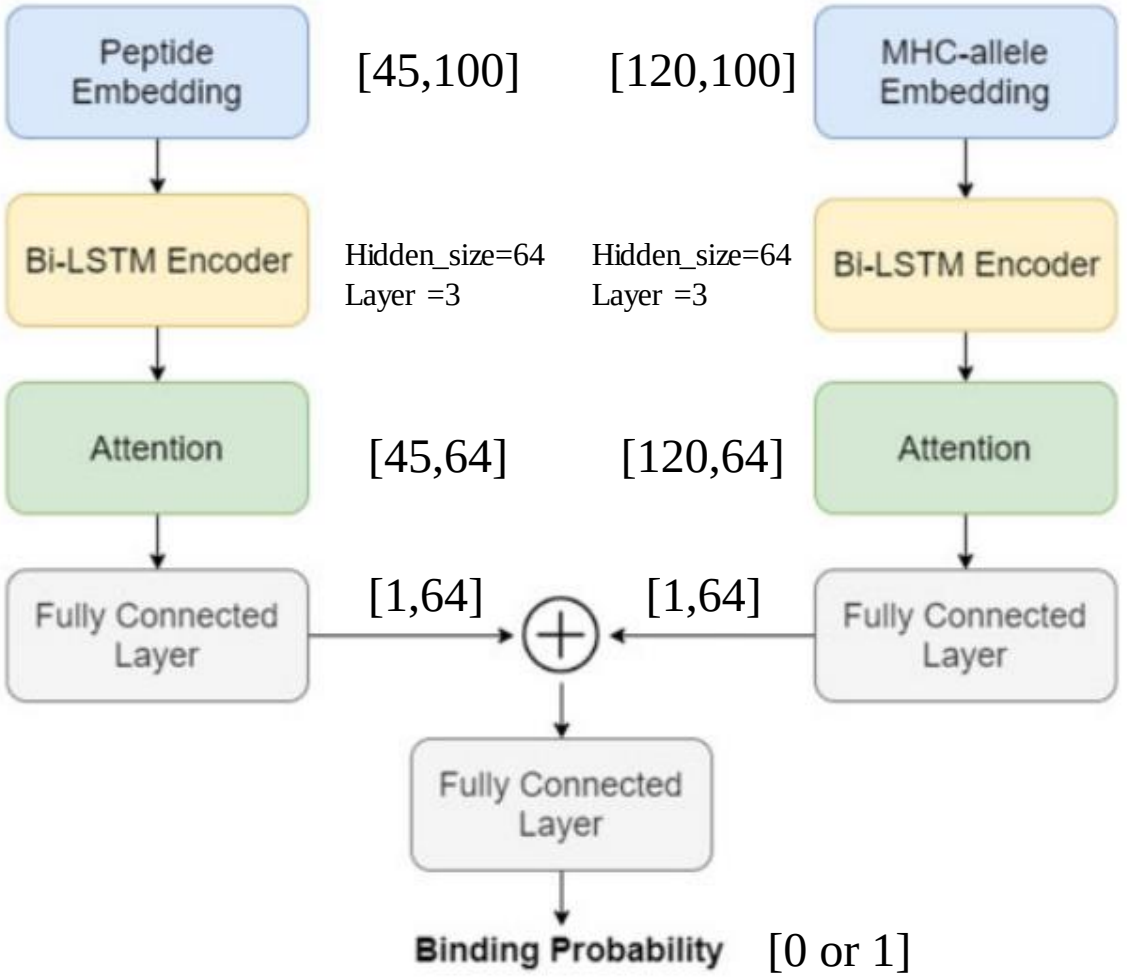
- MHC Allele: Class I (491,018) and Class II (64,954)
- Peptide: Any
- Binding score: "positive", "positive-high" and "positive-intermediate" (IC50 < 500nm) --> 1
  "negative" and "positive-low" --> 0

MHC I : 379,783 -> binding, 111,235 -> non-binding
MHC II: 36 035 -> binding, 28,919-> non-binding

Step 3: Embeddings of Peptides and MHC Alleles

- Peptide (shorter amino acid sequence) --> 1-gram --> max_len==45 --> [45, 100]
- MHC Allele (longer amino acid sequence) --> 3-gram --> max_len==120 --> [120, 100]

MHCAttnNet - Model



Train Details:
Optimizer: Adam
Learning rate: 0.001
Batch size: 32
Epoch: 25
5-fold cross-validation

MHCAttnNet - Model

Table 2. Comparison of performance of MHCAttnNet with the state-of-the-art methods for class I MHC alleles

Model	AUC-ROC	AUC-PRC	PCC	PPV	*F*1-score	Sensitivity
NetMHC4.0	0.8237	0.9048	0.5738	0.9486	0.8531	0.7757
MHCflurry	0.8234	0.9074	0.5624	**0.9695**	0.8230	0.7149
PUFFIN	0.8185	0.9129	0.5398	0.9668	0.8264	0.7215
MHCAttnNet (no attention)	0.8822	0.9380	0.7463	0.9643	0.9402	0.9303
MHCAttnNet	**0.8893**	**0.9418**	**0.7570**	0.9683	**0.9422**	**0.9304**

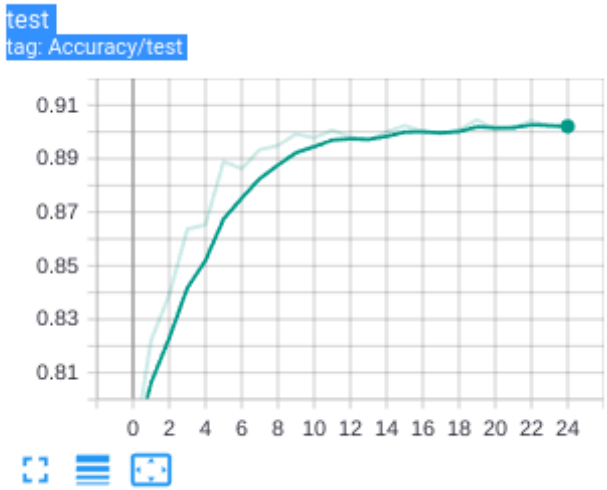
Note: The best performances are indicated in bold.

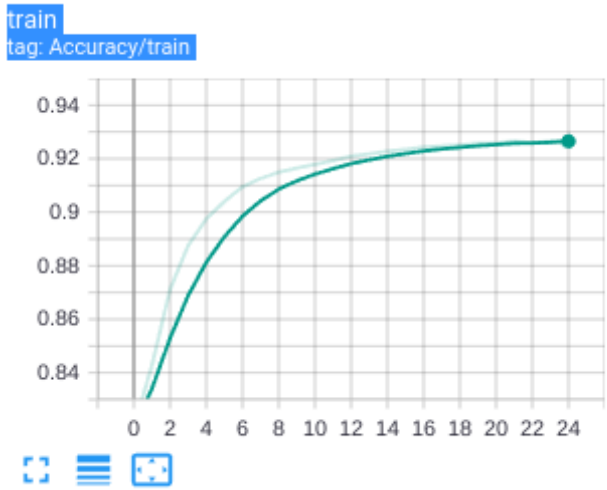
Table 3. Comparison of performance of MHCAttnNet with the state-of-the-art methods for class II MHC alleles

Model	Accuracy	AUC-ROC	PCC	Sensitivity	PPV	*F*1-score
NetMHCIIpan	0.6822	0.6751	0.4297	0.3888	0.9039	0.5435
PUFFIN	**0.7657**	**0.7756**	**0.5520**	0.6853	0.8644	0.7645
MHCAttnNet (no attention)	0.7503	0.7530	0.5032	0.7275	0.8732	0.7636
MHCAttnNet	0.7549	0.7579	0.5130	**0.7298**	**0.8769**	**0.7675**

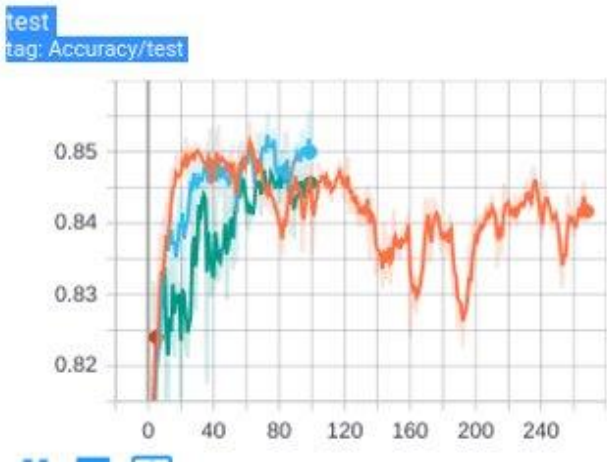
Note: The best performances are indicated in bold.

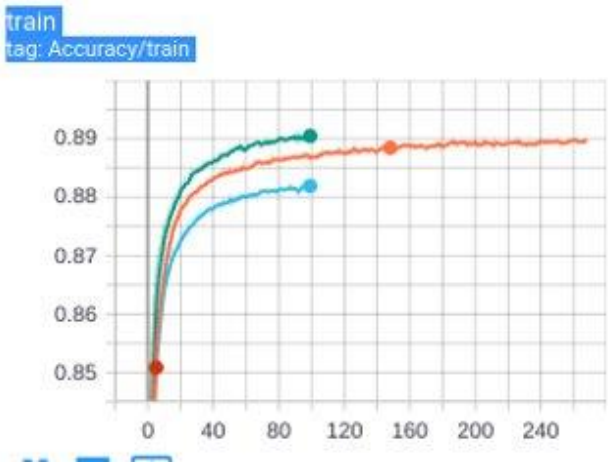
MHCAttnNet - LSTM





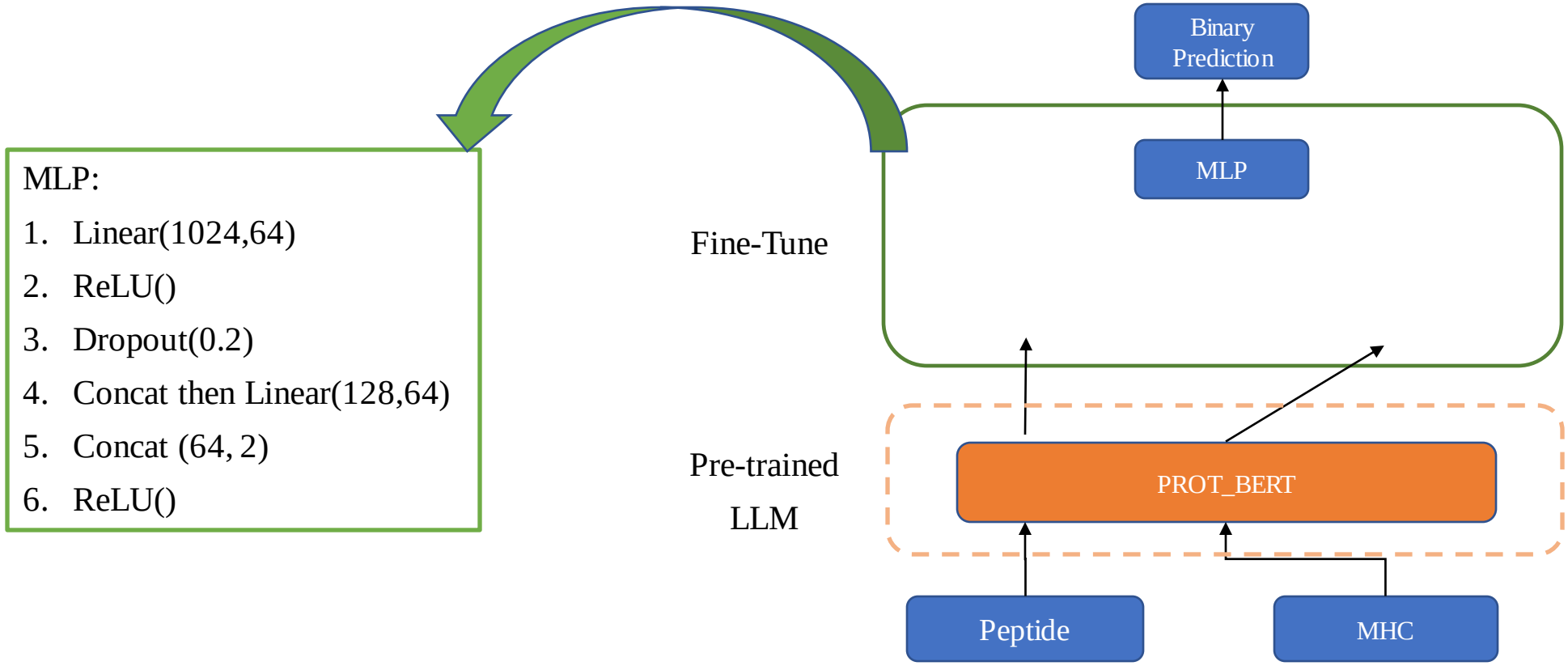
MHCAttnNet - Transformer





Task: Beat 3-layer Bi-LSTM for PEP-MHC Binding prediction

1. Using Pre-trained LLM model then fine-tune

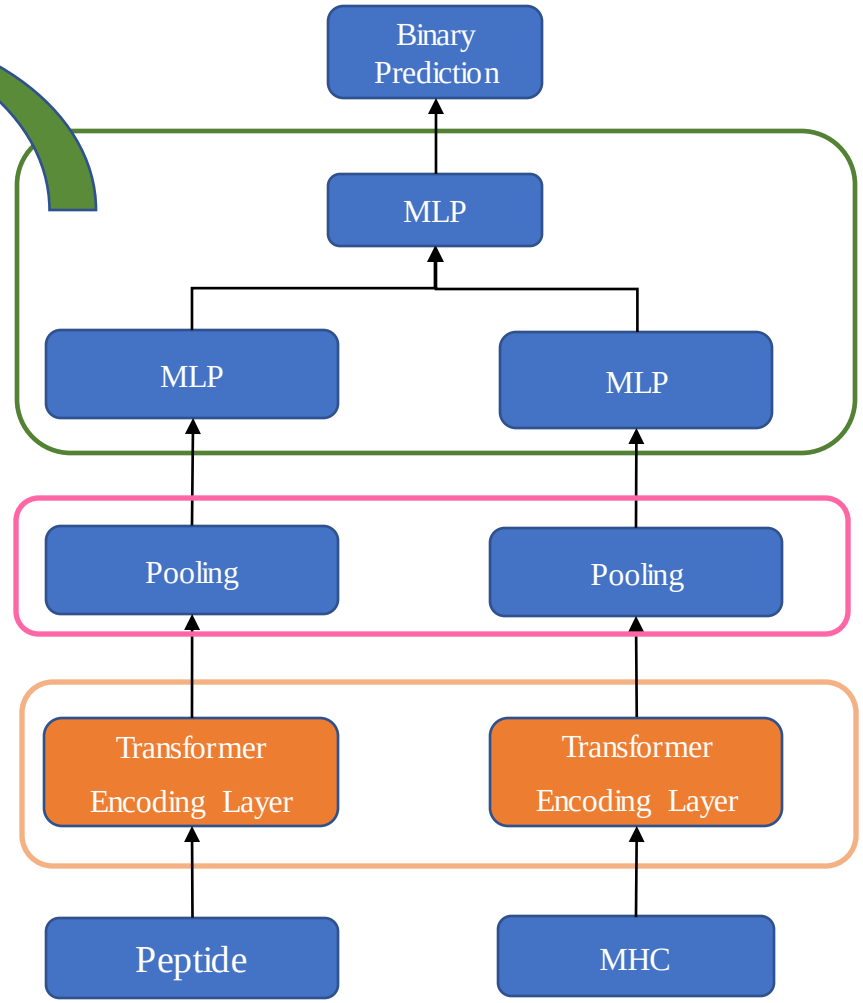


Task: Beat 3-layer Bi-LSTM for PEP-MHC Binding prediction

2. Train a light transformer from Scratch

MLP:

1. Flatten()
2. Mish()
3. Dropout(0.1)
4. Linear()
5. Mish()
6. Dropout(0.1)
7. Concat then Linear(128,64)
8. Mish()
9. Linear(64, 2)



Hyper-params may be Important When Training a Light Transformer

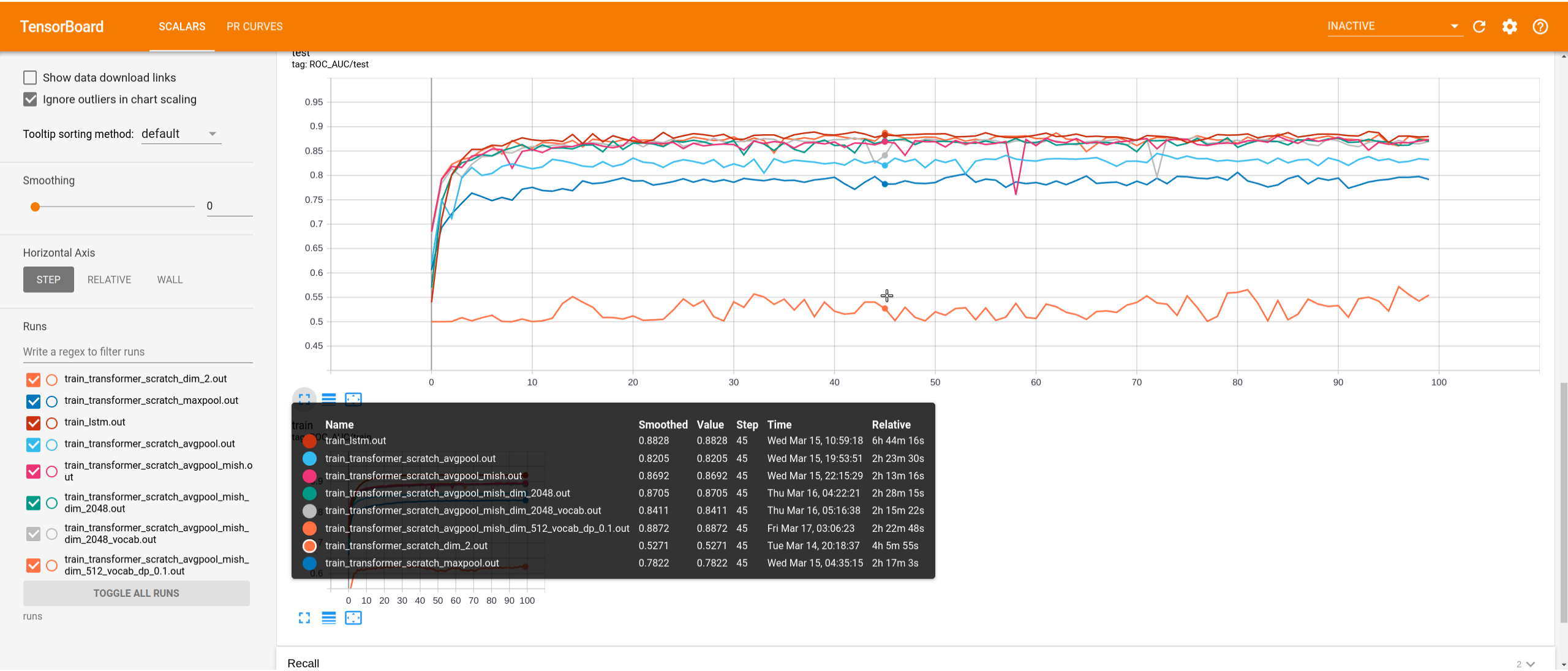
- # Encoder layer (e.g., **1**, 2, 3)
- # Head (e.g., **1**, 2, 10)
- # Forward dimension (e.g., 32, 64, 512, 2048)
- Dropout rate (**0.1**, **0.2**, 0.3)
- Pooling strategy (flatten, attention, mean on dim (batch, seq, emb), maxpool, **avgpool**)
- Vocab length (pos enc) (e.g., 1024, or = pep/mhc length)
- Activation in MLP (e.g., relu, selu, glu, **mish**)

Work towards to Approach LSTM Performance with Light Transformer

Model	#Trainable Params	# T-Layer	# T-Head	others	AUC-ROC	AUC-PRC	PCC	PPV	F1	Sensitivity
T-fine-tune	0.1M			Too Slow 30 BERT Layer (pre-trained) 419M (Non-trainable # params)	0.6936	0.8486	0.4359	0.8424	0.8872	0.9220
T-scratch-1	34M	2	1	Dim_2 Mean	0.5717	0.7996	0.2688	0.8217	0.8809	**0.9798**
T-scratch-2	34M	1	1	Max Pool	0.8030	0.8975	0.6619	0.9217	0.9298	0.9584
T-scratch-3	34M	1	1	Avg Pool	0.8448	0.9180	0.7172	0.9336	0.9388	0.9528
T-scratch-4	34M	1	1	Avg Pool ReLU->Mish (MLP)	0.8791	0.9356	0.7644	0.9453	0.9472	0.9504
T-scratch-5	35M	1	1	Avg Pool ReLU->Mish (MLP) Drop 0.2->0.1 (T)	0.8872	0.9401	**0.7696**	0.9492	**0.9473**	0.9449
3-layer Bi-LSTM (MHCAttNet)	0.6M	N/A	N/A	Original Paper Method (2020)	**0.8899**	**0.9425**	0.7499	**0.9687**	0.9393	0.9226

Example of ROC-AUC Testing Curve with Training Epochs

(Transformers with Different Settings vs. 3-Layer Attention based Bi-LSTM)



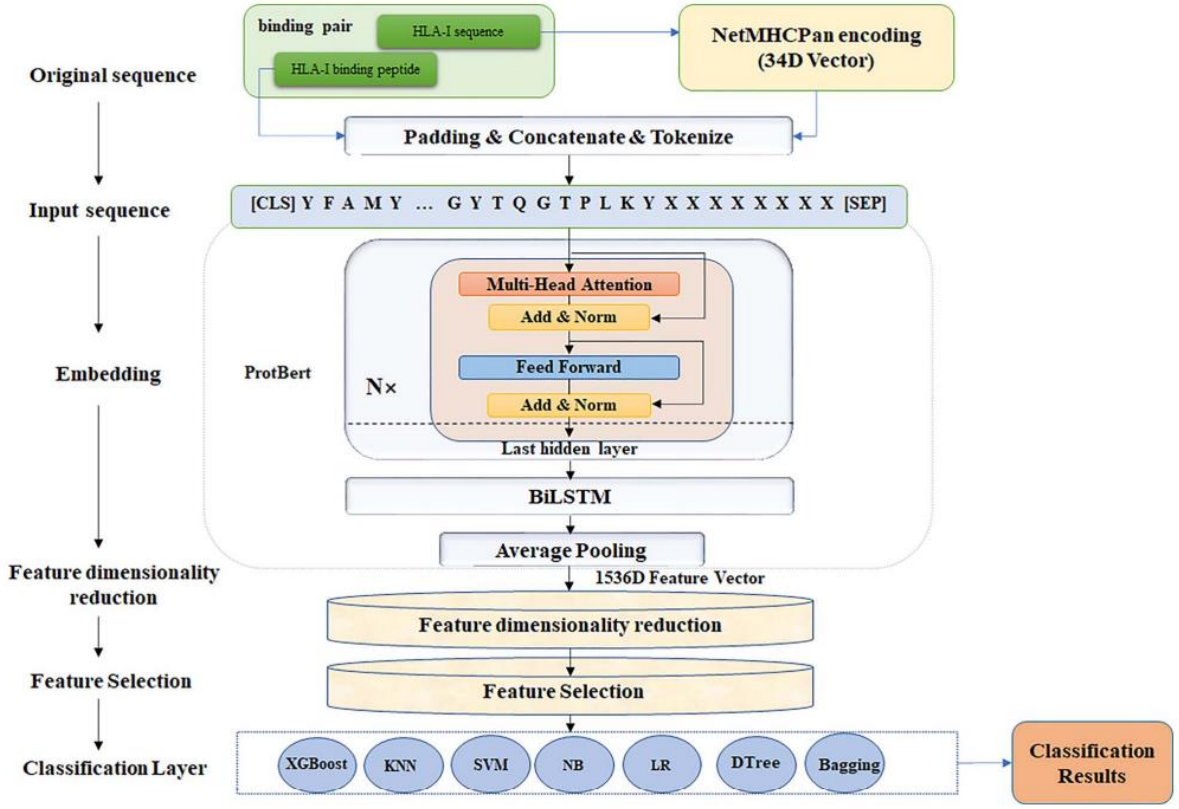
To Do List

Try to beat 3-layer LSTM with transformer

Try to figure out the fine-tuning

Read and implement more paper only on pep-mhc binding

- Input (e.g., protein seq)
- Encoding method (pseudo sequence, word2vec, pre-trained LLM)
- Model (e.g., Transformer, MLP fine-tuning)



Fine-Tuning/ Retrain

Model	#Trainable Params	# T-Layer	# T-Head	others	AUC-ROC	AUC-PRC	PCC	PPV	F1	Sensitivity
Prot-Trans	419M All parameters Trainable			12 hours/ epoch, total 10 epochs	**0.9249**	**0.9596**	**0.84486**	0.9669	**0.9646**	**0.9622**
3-layer Bi-LSTM (MHCAttNet)	0.6M	N/A	N/A	Original Paper Method (2020)	0.8899	0.9425	0.7499		0.9393	0.9226

Pep-MHC-I: **491018**
Unique Pep: **198607**
Unique MHC-I: **152**

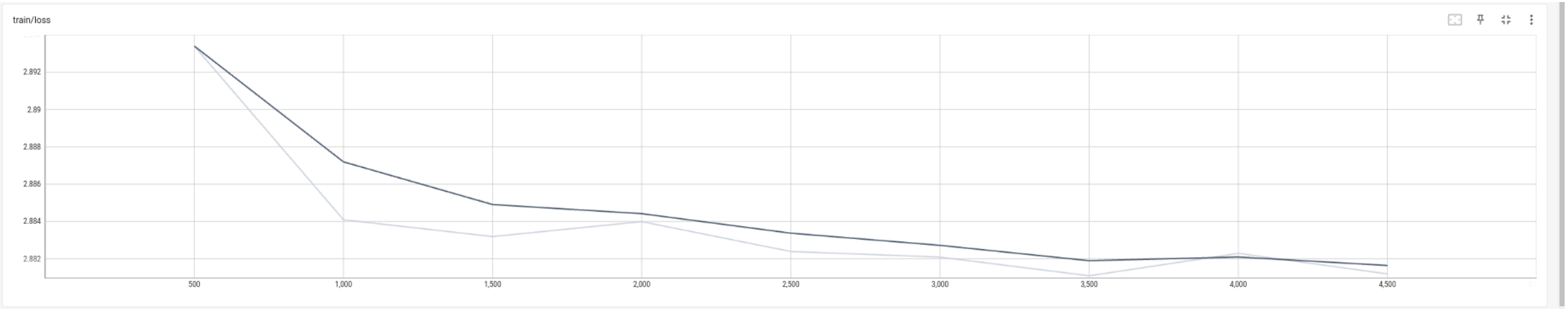
Fine-Tuning/ Retrain: New MHC Split

Model	#Trainable Params	# T-Layer	# T-Head	others	AUC-ROC	AUC-PRC	PCC	PPV	F1	Sensitivity
Prot-Trans	419M All parameters Trainable			12 hours/ epoch, total 2-5 epochs	**0.8055**	**0.8915**	**0.6400**	0.9006	**0.9178**	**0.9357**
3-layer Bi-LSTM (MHCAttNet)	0.6M	N/A	N/A	Original Paper Method (2020) 100 epochs	0.7739	0.8756	0.5813		0.9057	0.9277

Pep-MHC-I: **80-20 Split**
Unique MHC-I: **121- 30**

Pan Peptide LLM Training

- Dataset: IEDB entire Peptide, # unique peptide 1M
- Use the pre-trained tokenizer from Prot-Trans
- Model: Bert (same as Prot-Trans), padding max len = 32
- Task: Mask Language Model (15%)



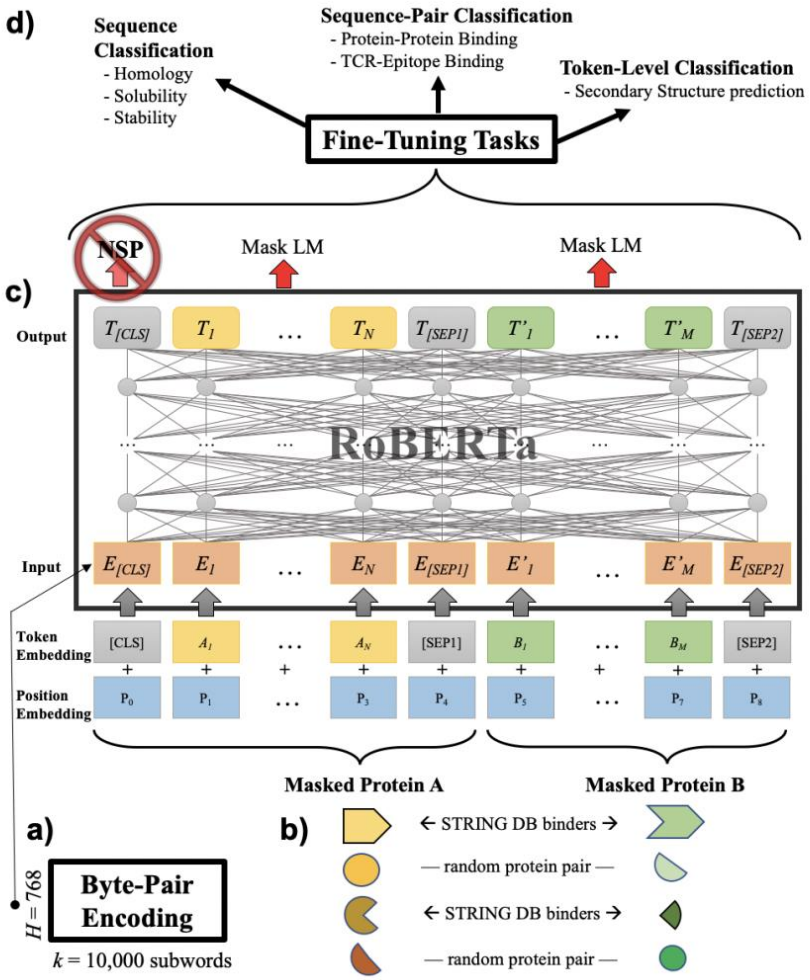
Fine-Tuning/Retrain on Pep-Trans

```
WARNING:torch.distributed.run:
*****************************************
Setting OMP_NUM_THREADS environment variable for each process to be 1 in default, to avoid your system being overloaded, please further tune the variable for optimal performance in your application as needed.
*****************************************
loading configuration file /home/patrick3/projects/def-wanglab-ab/patrick3/output_directory/pep/checkpoint-4500/config.json
Model config BertConfig {
  "_name_or_path": "/home/patrick3/projects/def-wanglab-ab/patrick3/output_directory/pep/checkpoint-4500/",
  "architectures": [
    "BertForMaskedLM"
  ],
  "attention_probs_dropout_prob": 0.0,
  "classifier_dropout": null,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.0,
  "hidden_size": 1024,
  "initializer_range": 0.02,
  "intermediate_size": 4096,
  "layer_norm_eps": 1e-12,
  "max_position_embeddings": 40000,
  "model_type": "bert",
  "num_attention_heads": 16,
  "num_hidden_layers": 30,
  "pad_token_id": 0,
  "position_embedding_type": "absolute",
  "torch_dtype": "float32",
  "transformers_version": "4.26.0",
  "type_vocab_size": 2,
  "use_cache": true,
  "vocab_size": 30
}
```

```
  0%|          | 0/4070 [00:00<?, ?it/s][W reducer.cpp:1298] Warning: find_
[W reducer.cpp:1298] Warning: find_unused_parameters=True was specified in
[W reducer.cpp:1298] Warning: find_unused_parameters=True was specified in
[W reducer.cpp:1298] Warning: find_unused_parameters=True was specified in

  0%|          | 1/4070 [00:12<14:36:30, 12.92s/it]
  0%|          | 2/4070 [00:25<14:26:43, 12.78s/it]
  0%|          | 3/4070 [00:37<13:58:07, 12.36s/it]
  0%|          | 4/4070 [00:49<13:43:40, 12.15s/it]
  0%|          | 5/4070 [01:01<13:55:12, 12.33s/it]
  0%|          | 6/4070 [01:13<13:44:04, 12.17s/it]
  0%|          | 7/4070 [01:25<13:36:34, 12.06s/it]
  0%|          | 8/4070 [01:37<13:32:31, 12.00s/it]
  0%|          | 9/4070 [01:49<13:28:51, 11.95s/it]
  0%|          | 10/4070 [02:01<13:26:12, 11.91s/it]
  0%|          | 11/4070 [02:13<13:26:35, 11.92s/it]
  0%|          | 12/4070 [02:24<13:24:46, 11.90s/it]
  0%|          | 13/4070 [02:36<13:23:21, 11.88s/it]
  0%|          | 14/4070 [02:48<13:22:28, 11.87s/it]
  0%|          | 15/4070 [03:00<13:21:41, 11.86s/it]
  0%|          | 16/4070 [03:12<13:32:58, 12.03s/it]
  0%|          | 17/4070 [03:24<13:29:20, 11.98s/it]
  0%|          | 18/4070 [03:37<13:35:05, 12.07s/it]
  0%|          | 19/4070 [03:48<13:30:52, 12.01s/it]
  0%|          | 20/4070 [04:00<13:27:24, 11.96s/it]
  1%|          | 21/4070 [04:12<13:24:48, 11.93s/it]
  1%|          | 22/4070 [04:25<13:37:21, 12.12s/it]
```

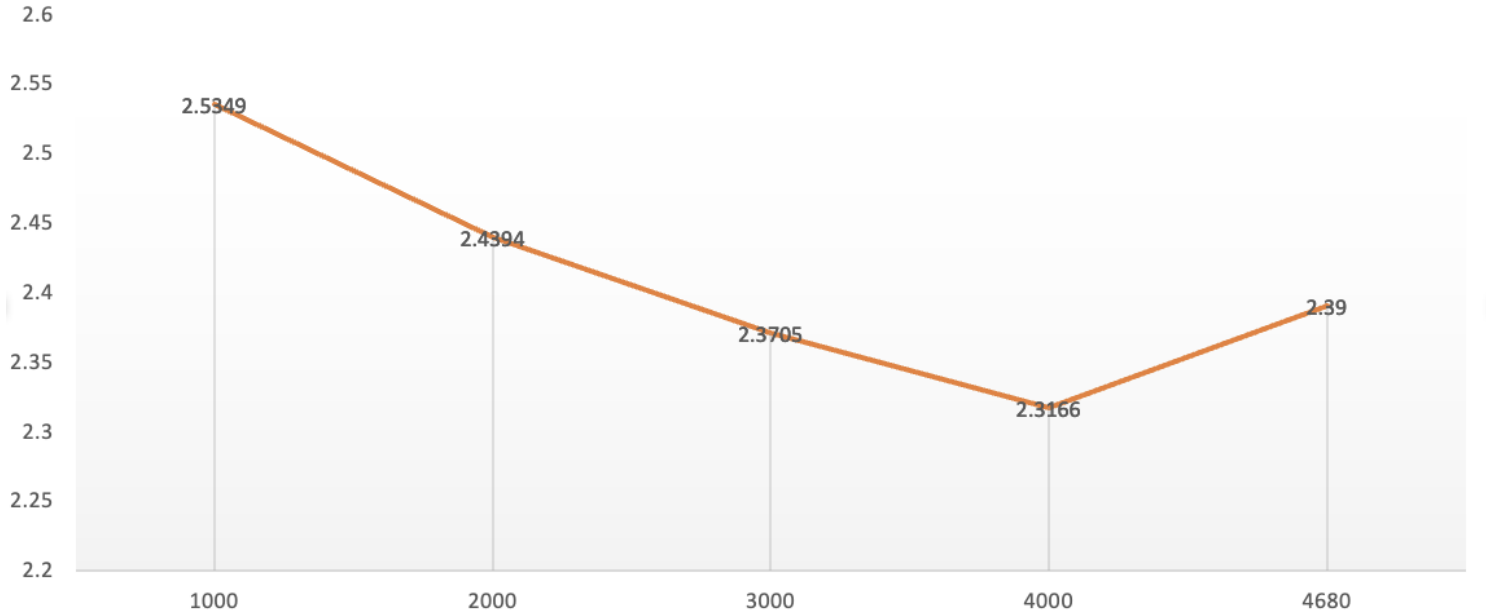
Sentence Pair - Pretraining



Pep max len 32
MHC max len: 256
Trans max len?

Input : Pep X MHC

Peptide Pretraining



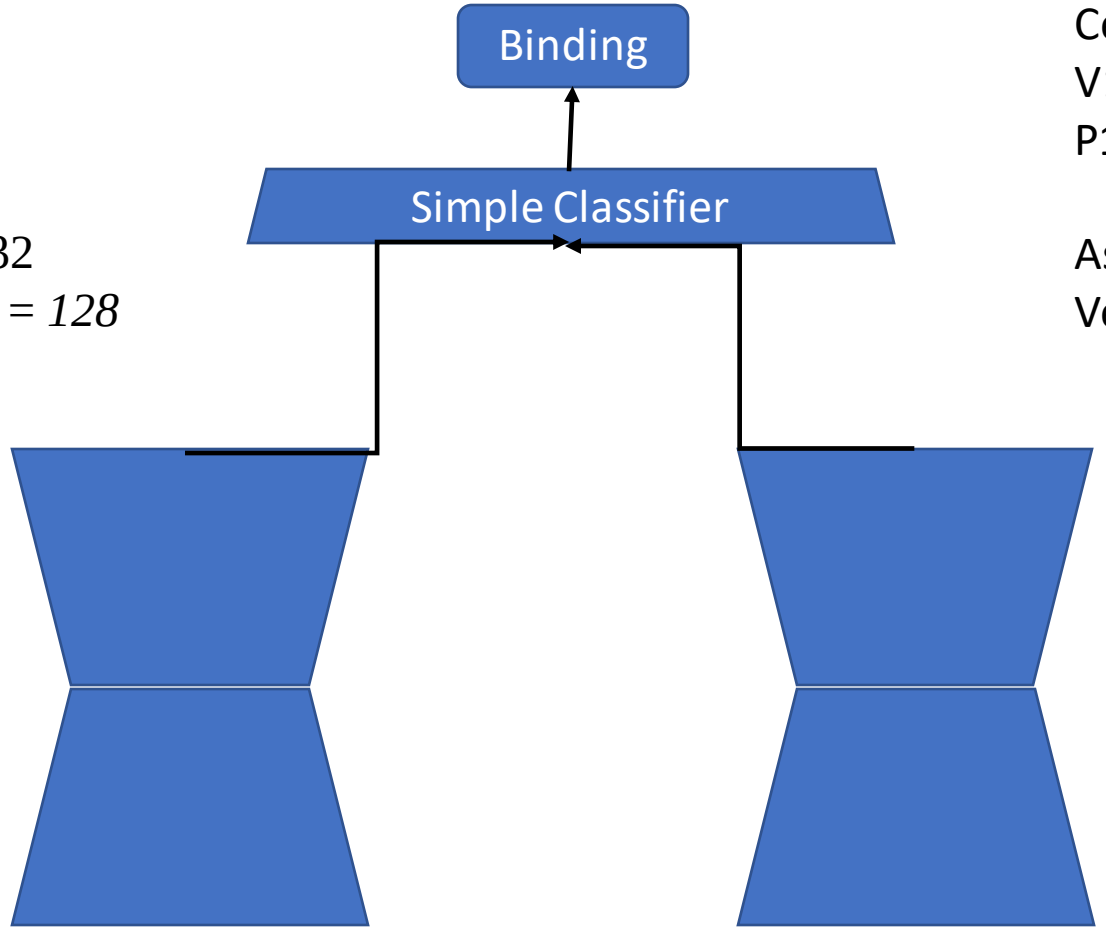
Prot-Bert: 419M
Samples: 959749
LLM : 25%
Epochs: 10
Batch per device: 8
GPU: 4
Gradient accumulation steps: 64
Total train batch size: 8x4x 64 = *2048*
*Total optimization steps: (*959749 x 10)/ 4680 =4680
Training time: 16hrs

Our Model

	Module 1	Module 2
Architecture	BERT	BERT
Tokenizer	PROT-BERT	PROT-BERT
Pre-training	Peptide Only (IEDB, =1M)	Protein (Uniref 100, >1B)
Pretrain Seq max length	32	2048
Pre-trained Task	MLM (25%)	Many
Use Pretrained weight	YES (Ours)	Yes (Prot-Bert)
Downstream Task		
Our Input	Peptide	MHC- I
Our max seq length	32	256

Our Model

Params: *838 M*
Samples: 491018
Epochs: 10
Batch per device: 2
GPU: 2
Gradient accumulation steps: 32
Total train batch size: 2x2x 32 = *128*
Total optimization steps: 3836
Training time: 50 hrs
GPU Mem: 15G



Need higher GPU memory for faster training
Cedar:
V100: 16G
P100: 12G

Asked, Vivian, will check gpus on Vector

Constraints on LLM

1. CCDB: still job stop reason not found. CCDB support asked me to change the bash script commands, but more problems occurs.
2. Vector Institute: Hard to use Vivian's credentials because MFA code from her phone required each time log in
3. Chat with Laura, H4H is the only option. --> Chat with Zhibin, cannot assign GPUs to me for a couple of days since H4H is only job submission-based.
4. What's next? Colab pro + ? Change back to use single pre-trained model or train single LLM from scratch? Find new innovation points and focus more on analysis.

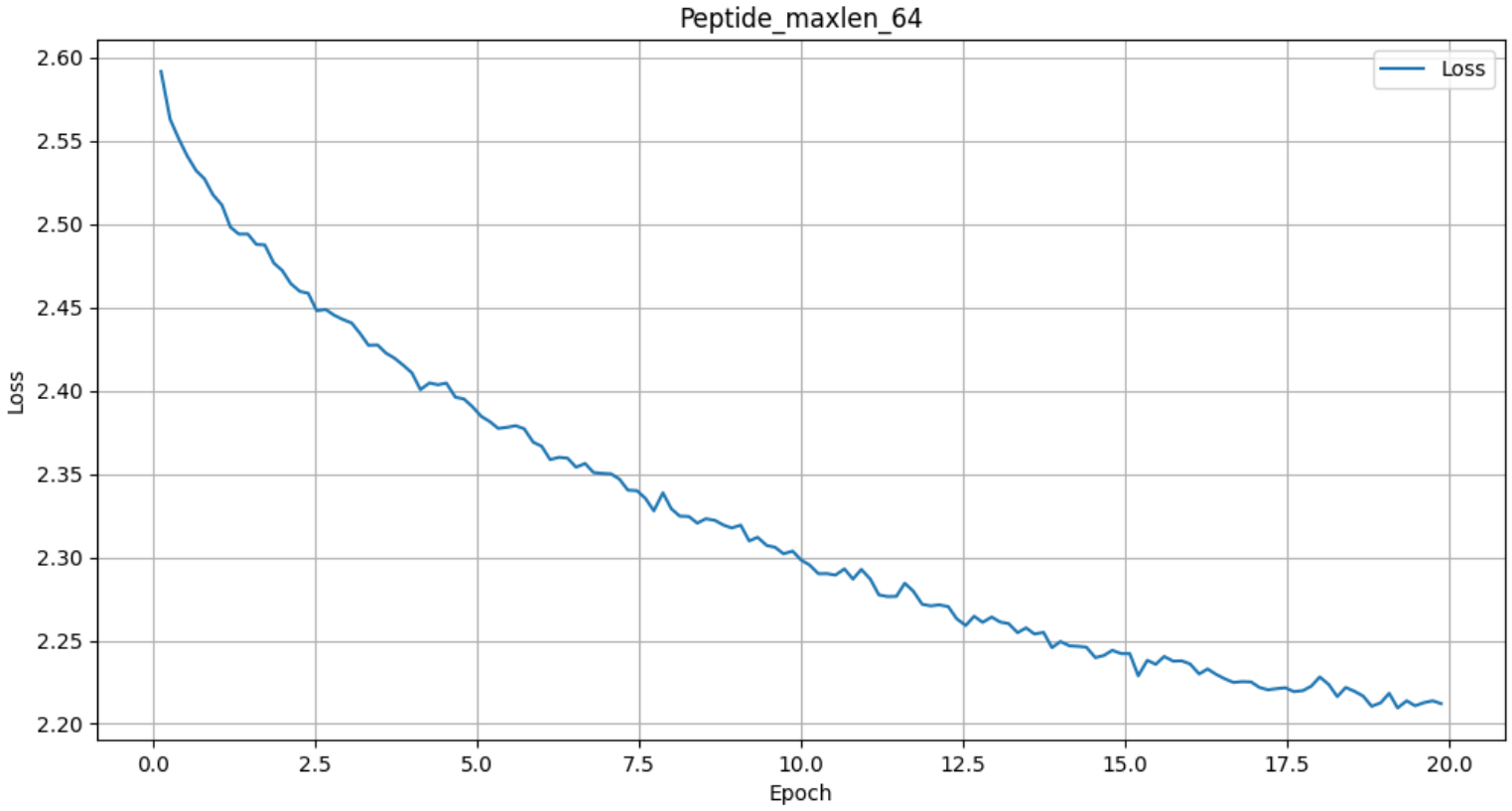
Current work

- Help Chengpeng from Hansen's Lab on CircRNA

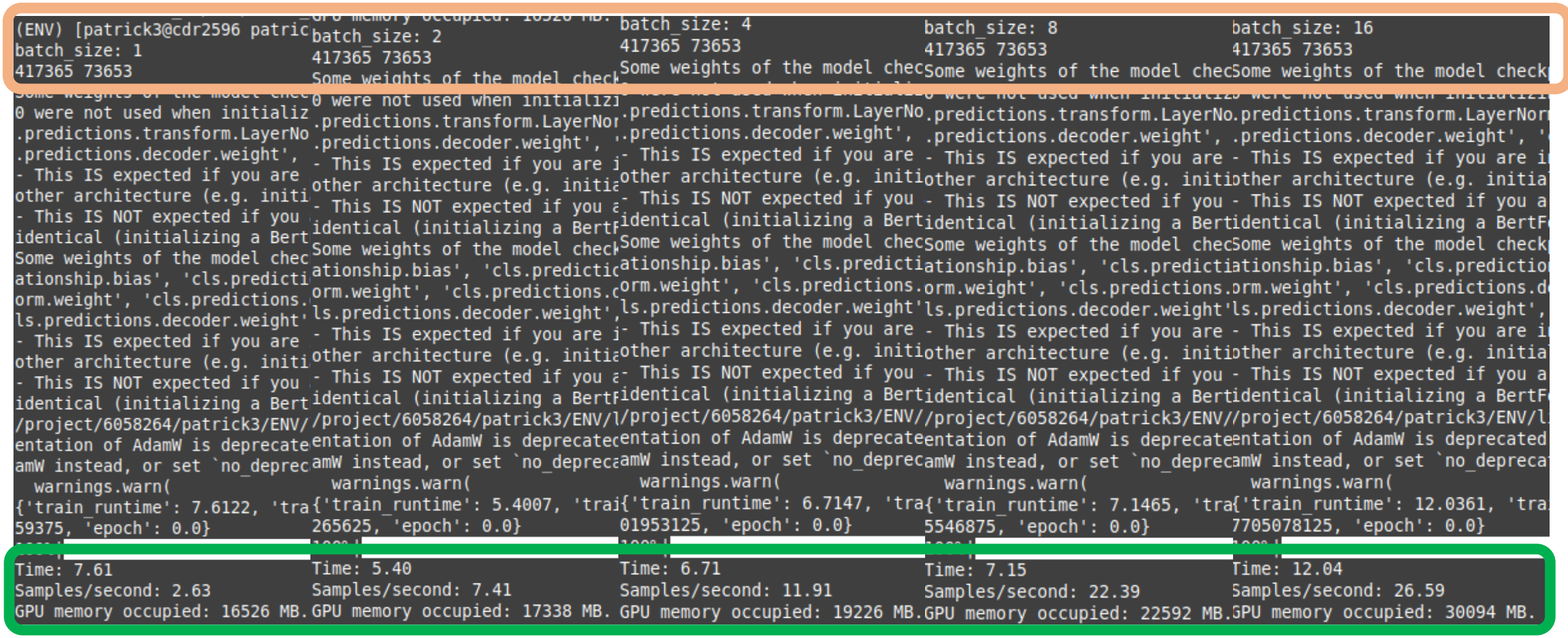
1. Construct new database according to CircRNA start and end point, from Genome Ref 38. (Done)
2. Find common sequences in the upstream (200) of the sequence while they have complimentary part in the downstream(200) (Done)
3. View sample distribution on circular rate and determine threshold
4. Implement deep learning to discover which part (introns and exon) is the critical factor

- Participated in Standford Competition alone in my **spare** time and got the award.

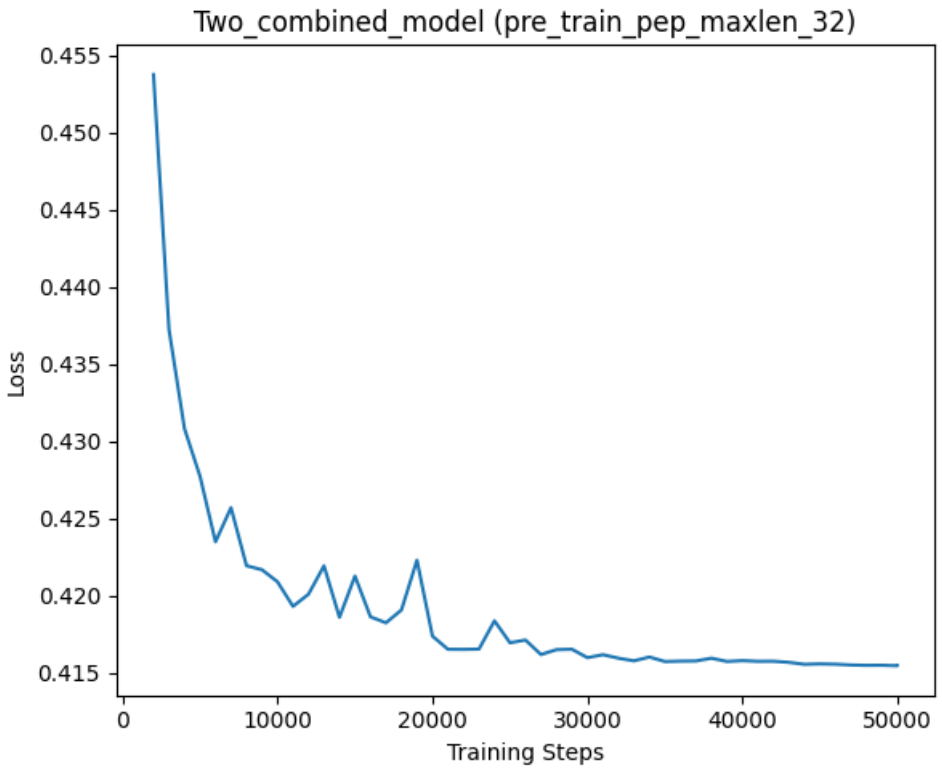
Pretraining peptide sequences (1M) on BERT

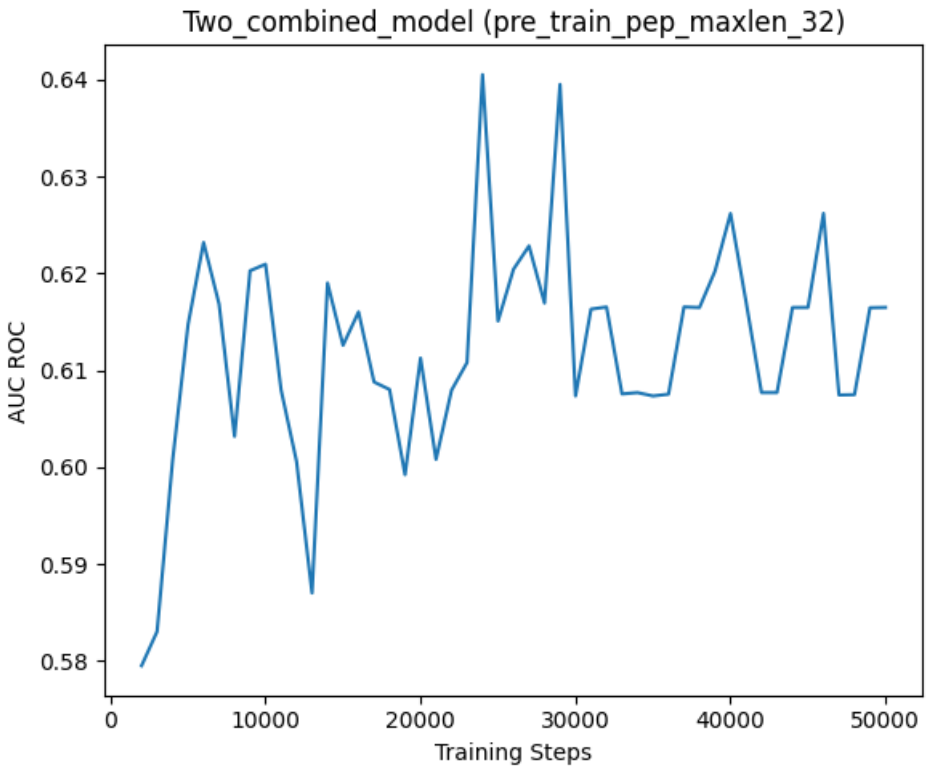


Two Cascade BERT (Memory Requirement)

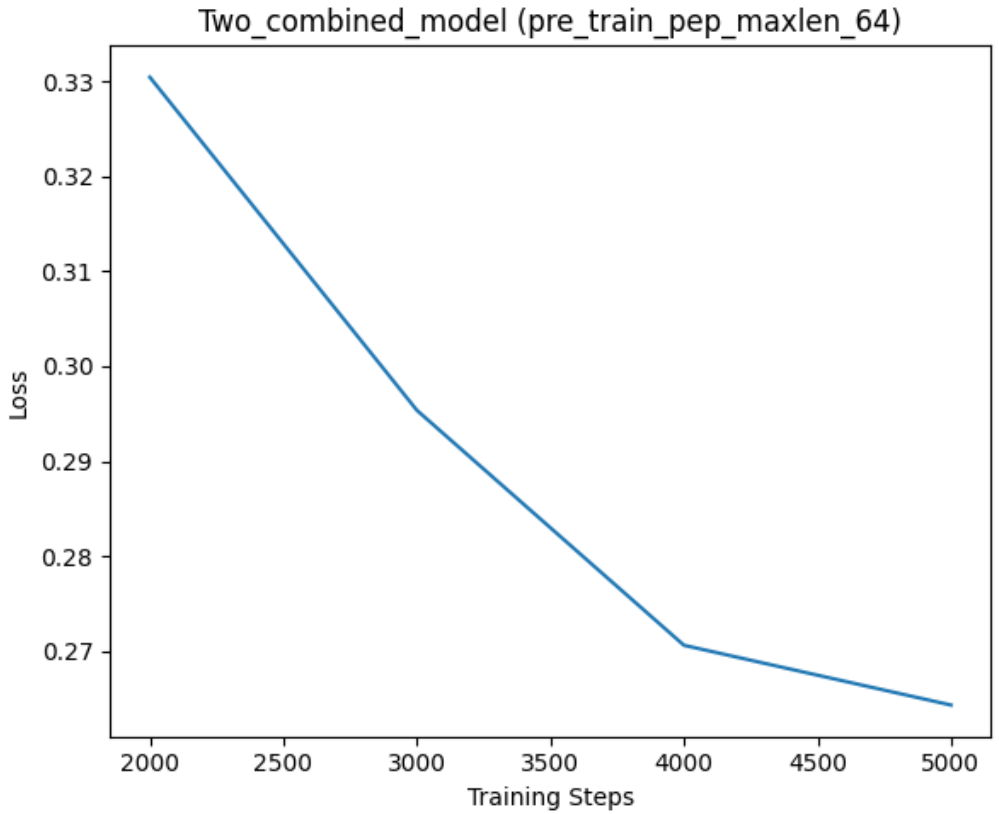


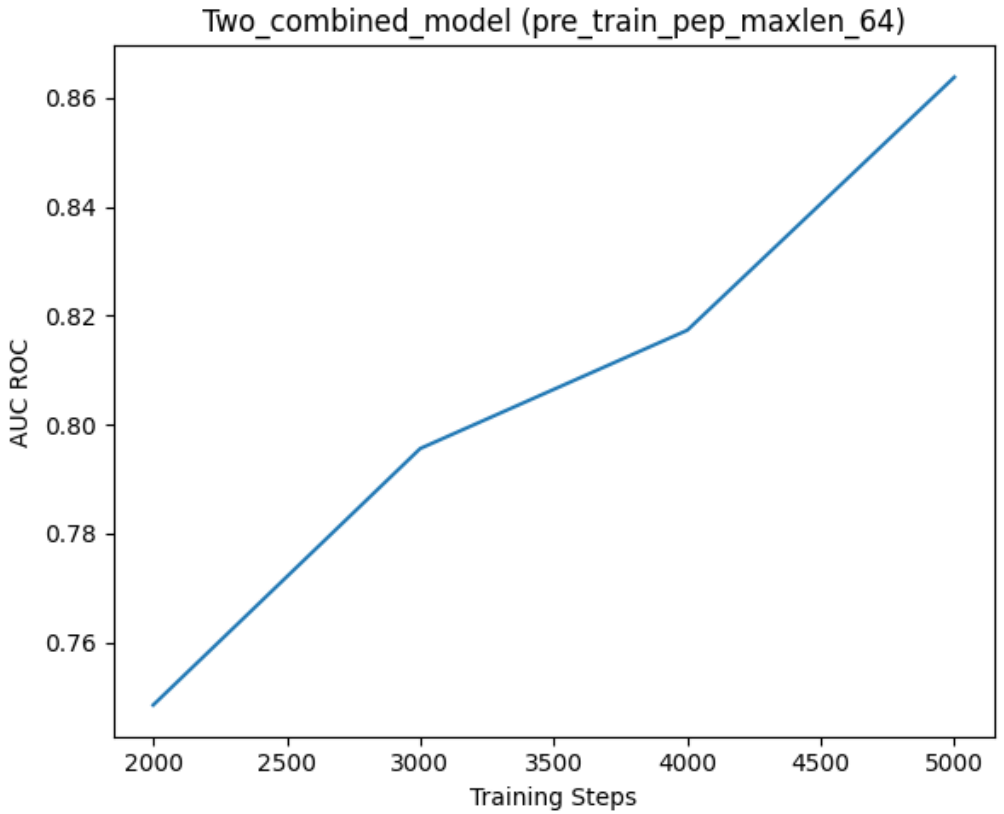
Pretrained Peptide BERT (input PEP) + Pretrained PROT_BERT (input MHC)





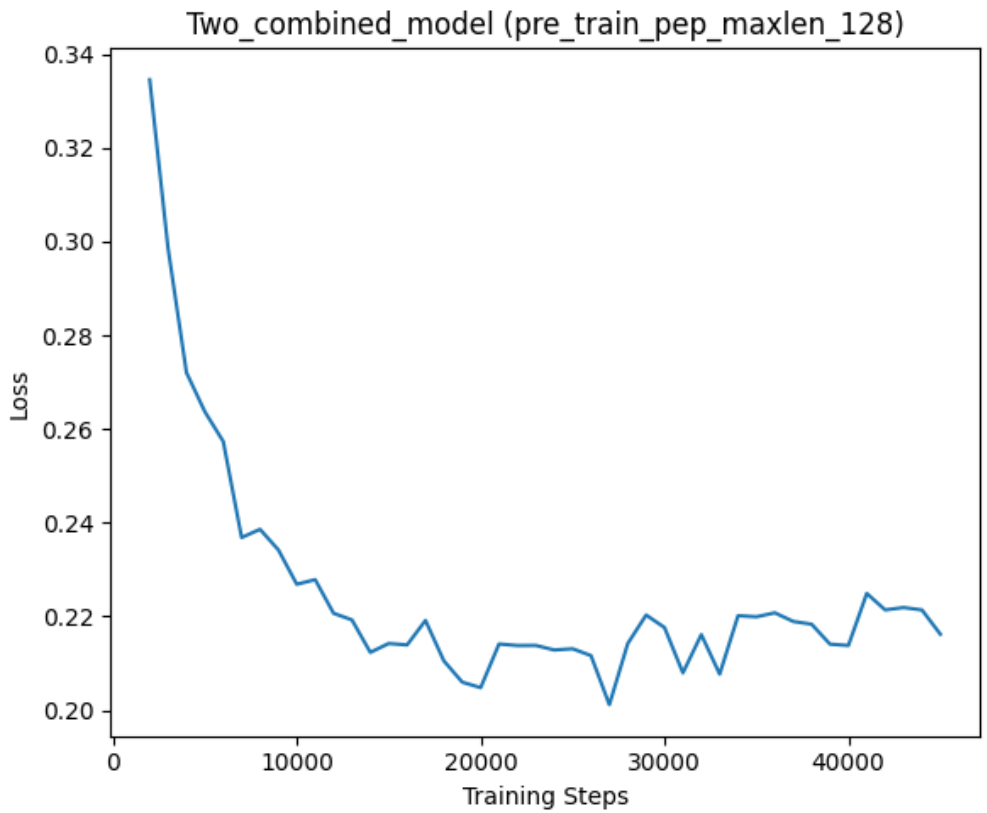
Pretrained peptide BERT (input PEP) + Pretrained Prot_BERT (input MHC)

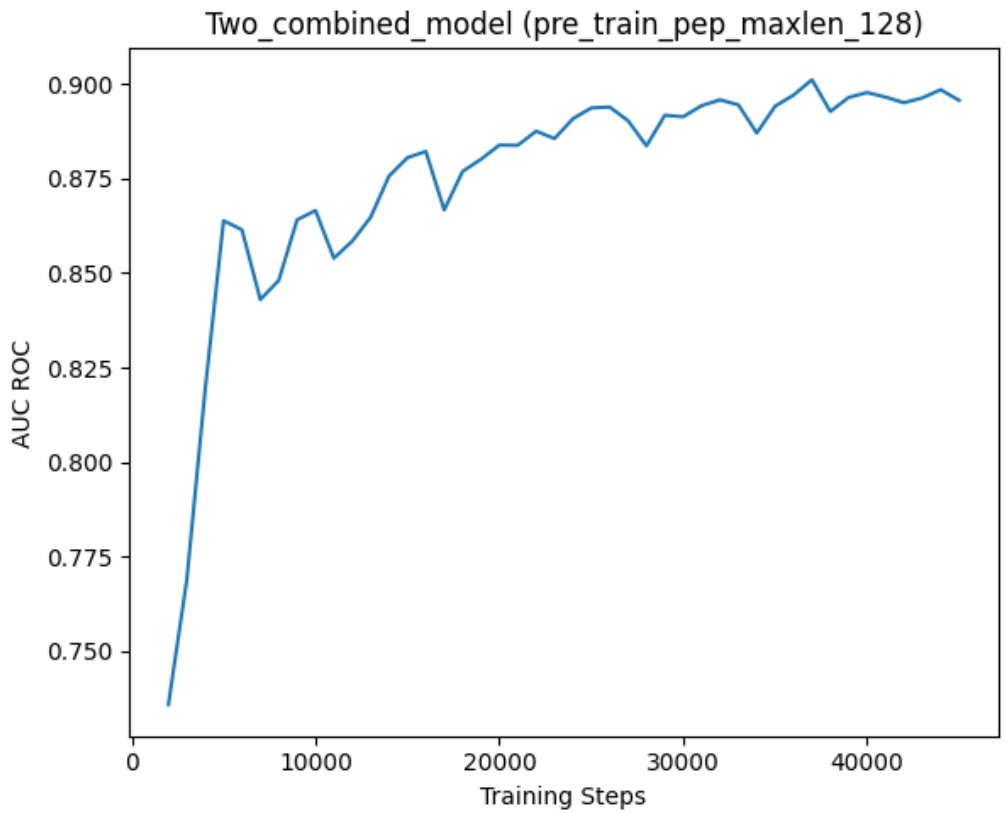




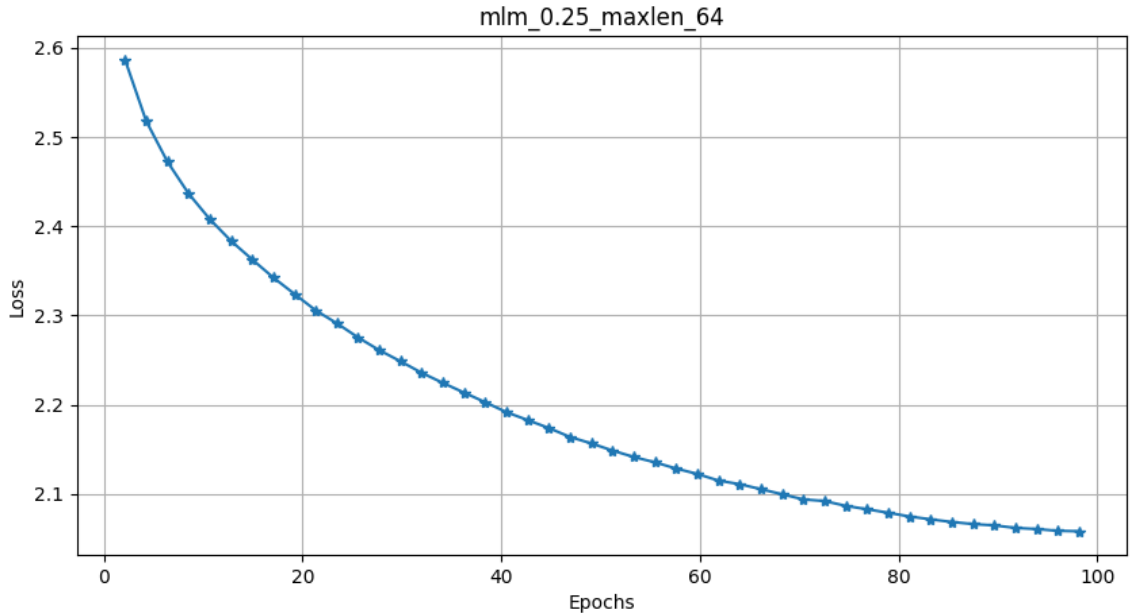

```
12%|          | 5761/50000 [4:54:37<27:56:29,  2.27s/it]
12%|          | 5762/50000 [4:54:39<27:43:39,  2.26s/it]
12%|          | 5763/50000 [4:54:41<27:37:18,  2.25s/it]
12%|          | 5764/50000 [4:54:44<27:32:11,  2.24s/it]
12%|          | 5765/50000 [4:54:46<27:29:41,  2.24s/it]
12%|          | 5766/50000 [4:54:48<27:27:42,  2.23s/it]
12%|          | 5767/50000 [4:54:50<27:29:47,  2.24s/it]
12%|          | 5768/50000 [4:54:53<27:26:46,  2.23s/it]
12%|          | 5769/50000 [4:54:55<27:25:08,  2.23s/it]
12%|          | 5770/50000 [4:54:57<27:24:43,  2.23s/it]
12%|          | 5771/50000 [4:54:59<27:23:36,  2.23s/it]
12%|          | 5772/50000 [4:55:02<29:28:04,  2.40s/it]
12%|          | 5773/50000 [4:55:04<28:49:25,  2.35s/it]
12%|          | 5774/50000 [4:55:06<28:22:03,  2.31s/it]/var/spool/slurmd/job1818382/slurm_script: line 21: 18872 Bus error               (core dumped) python3 /home/patrick3/projects/def-wanglab-ab/patrick3/MHCAttnNet_ft/src/train_protbert_ft_fast.py
slurmstepd: error: unlink(/var/spool/slurmd/job1818382/slurm_script): Input/output error
slurmstepd: error: rmdir(/var/spool/slurmd/job1818382): Input/output error
slurmstepd: error: Unable to unlink domain socket `/var/spool/slurmd/cdr2552_1818382.4294967291`: Input/output error
```

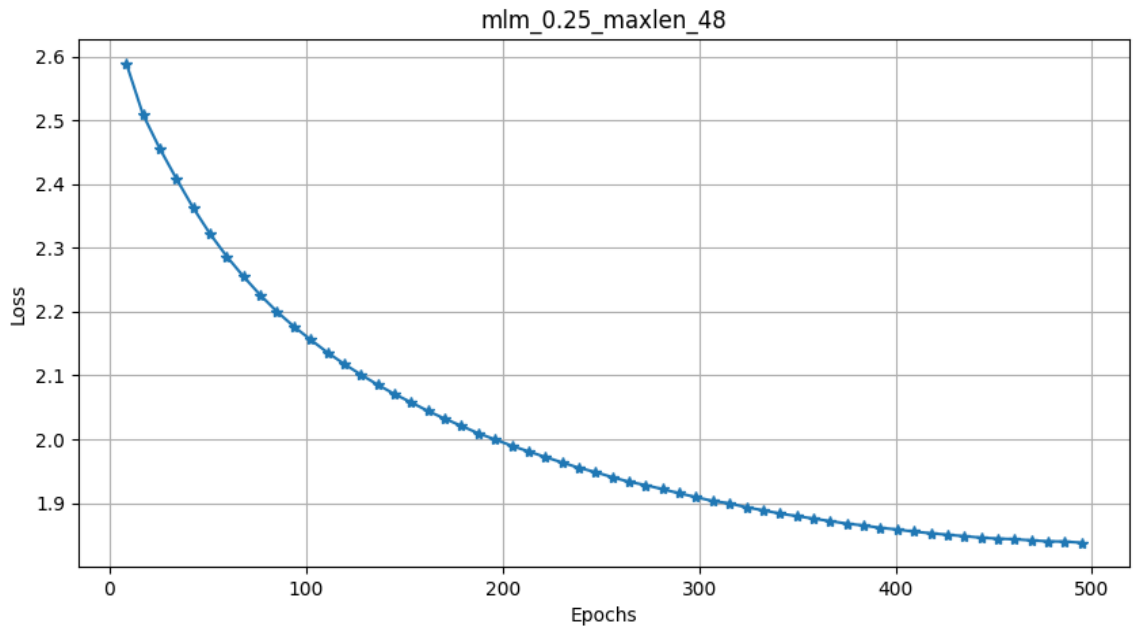
Pretrained peptide BERT (input PEP) + Pretrained PROT_BERT (input MHC)



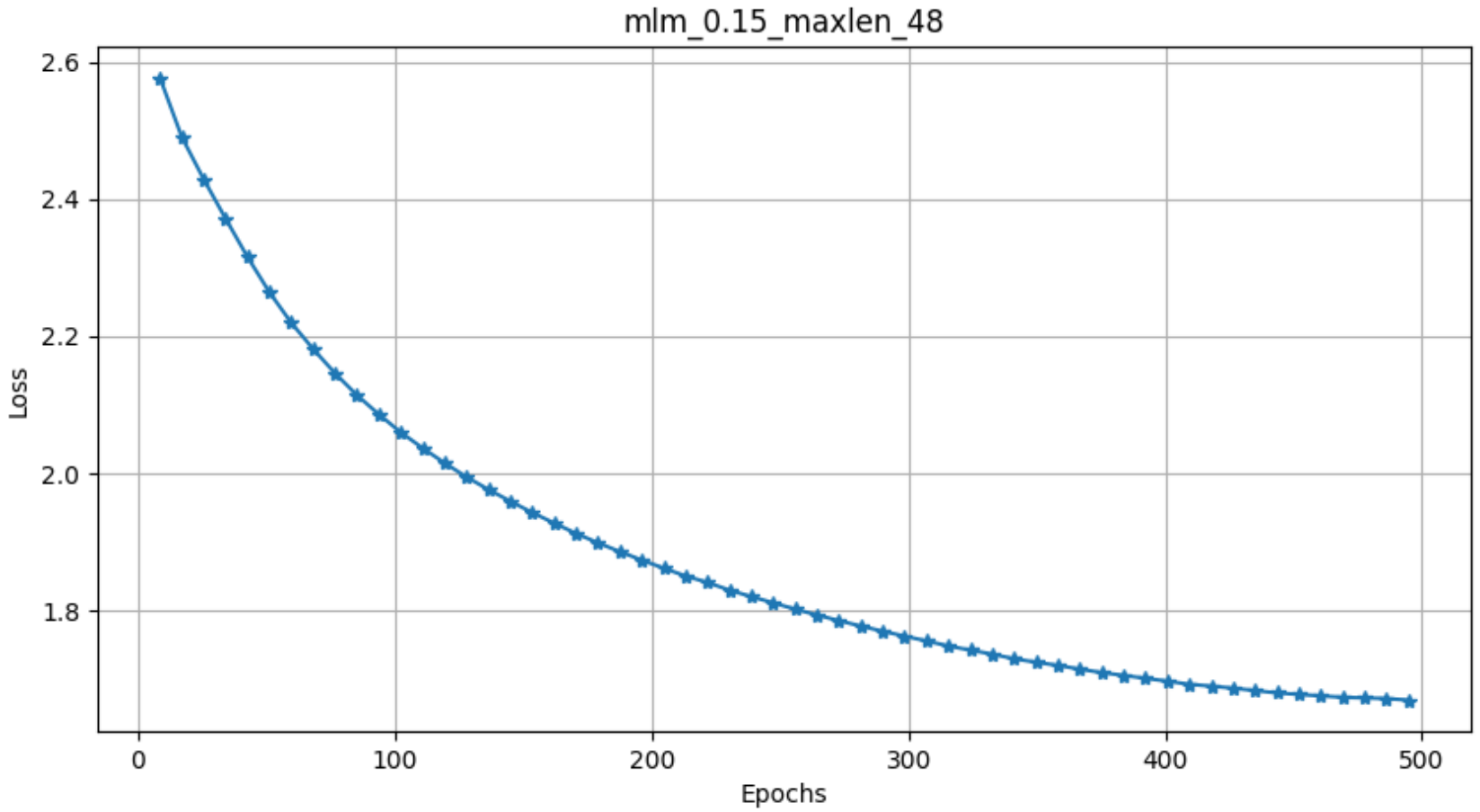


Pretrained peptide BERT (input PEP)

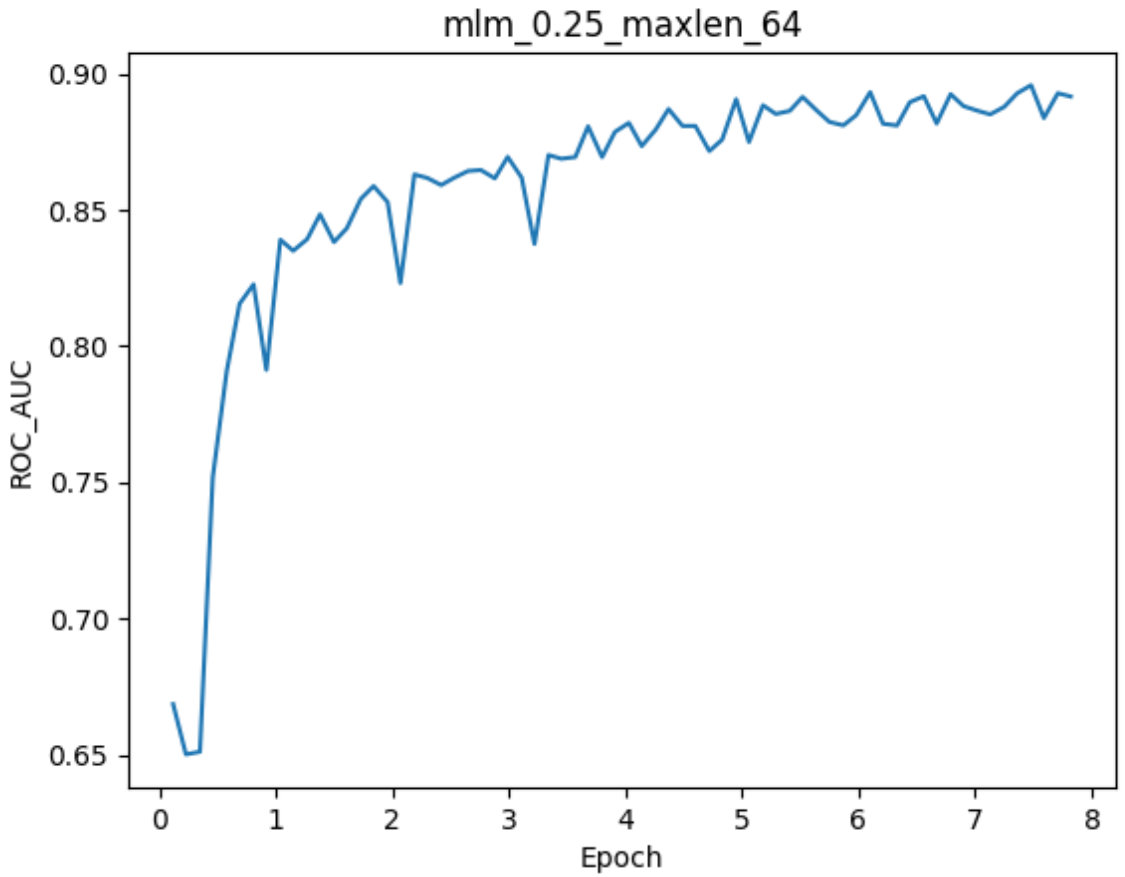


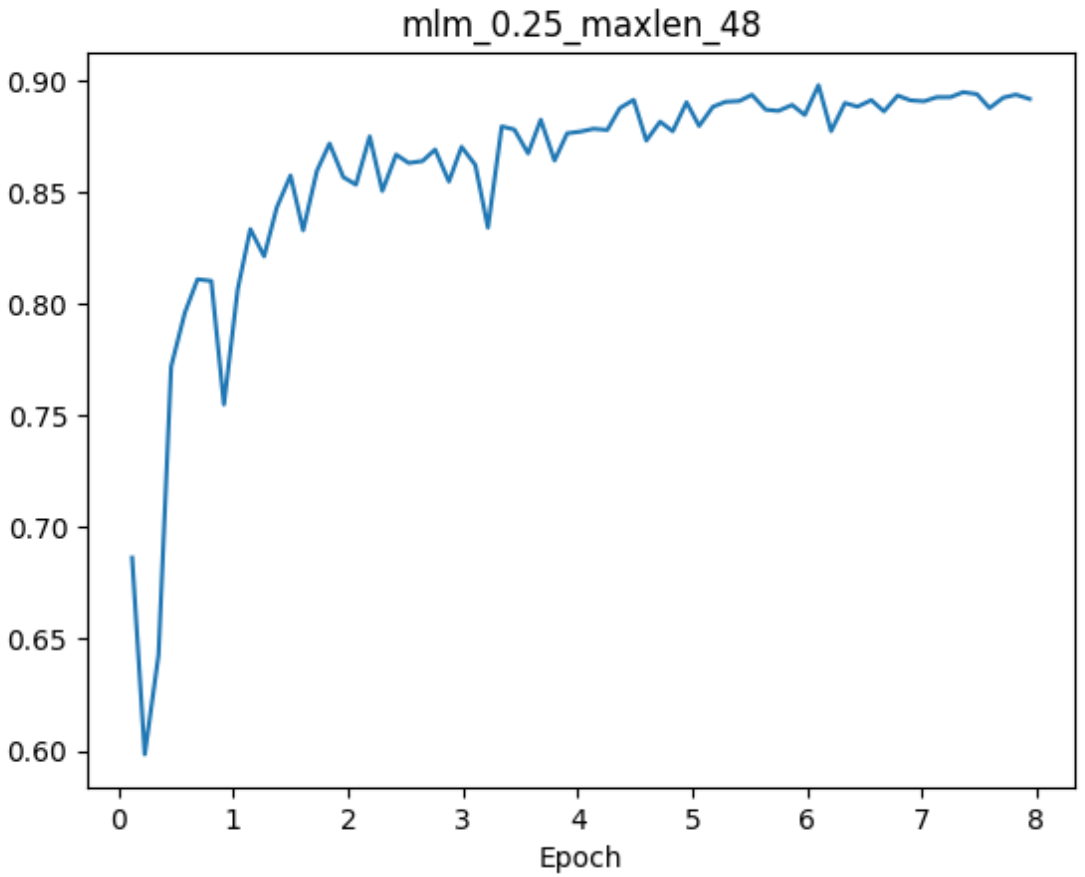


MLM ratio decrease?



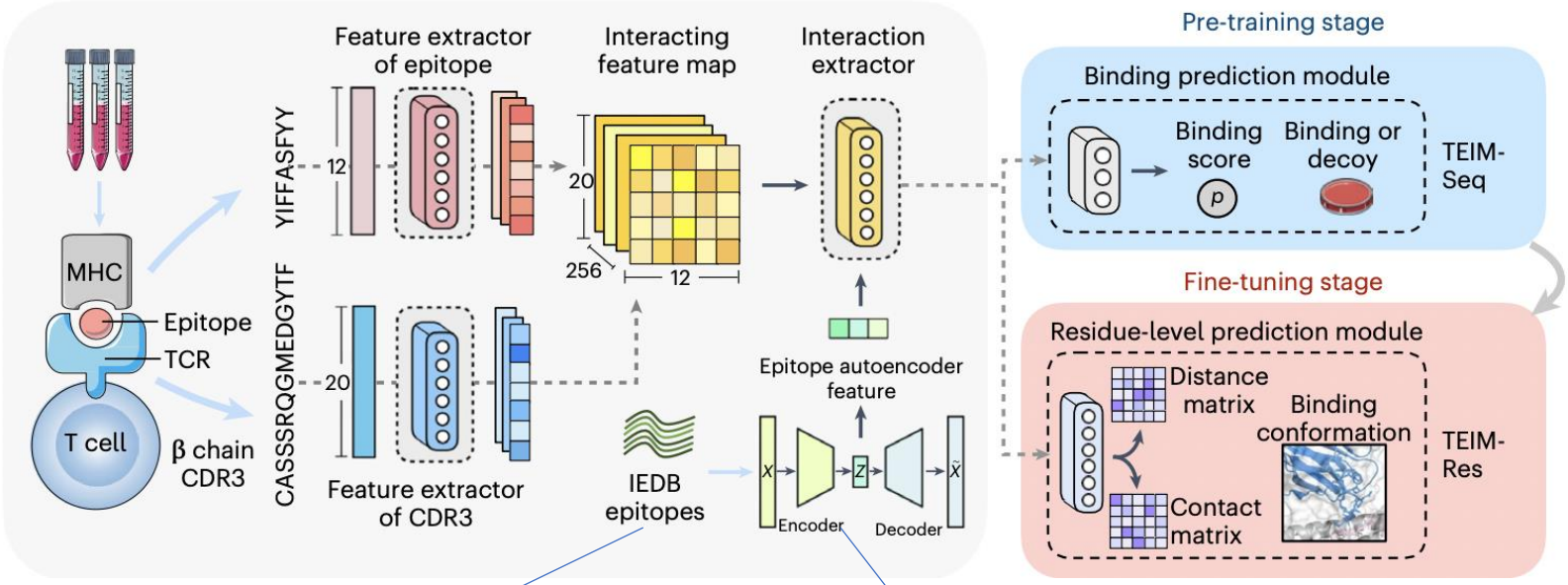
Pretrained peptide BERT (input PEP) + Pretrained Prot_BERT (input MHC)





T cell Receptor – Epitope Binding

Nat. Machine Intel Feb 2023



(1) Linear Sequence
(2) No B-cell Essay
(3) MHC I
(4) Len 8-12
(5) 450k unique sequence

[1,12] peptide

[20,20]

Dictionary/embedding

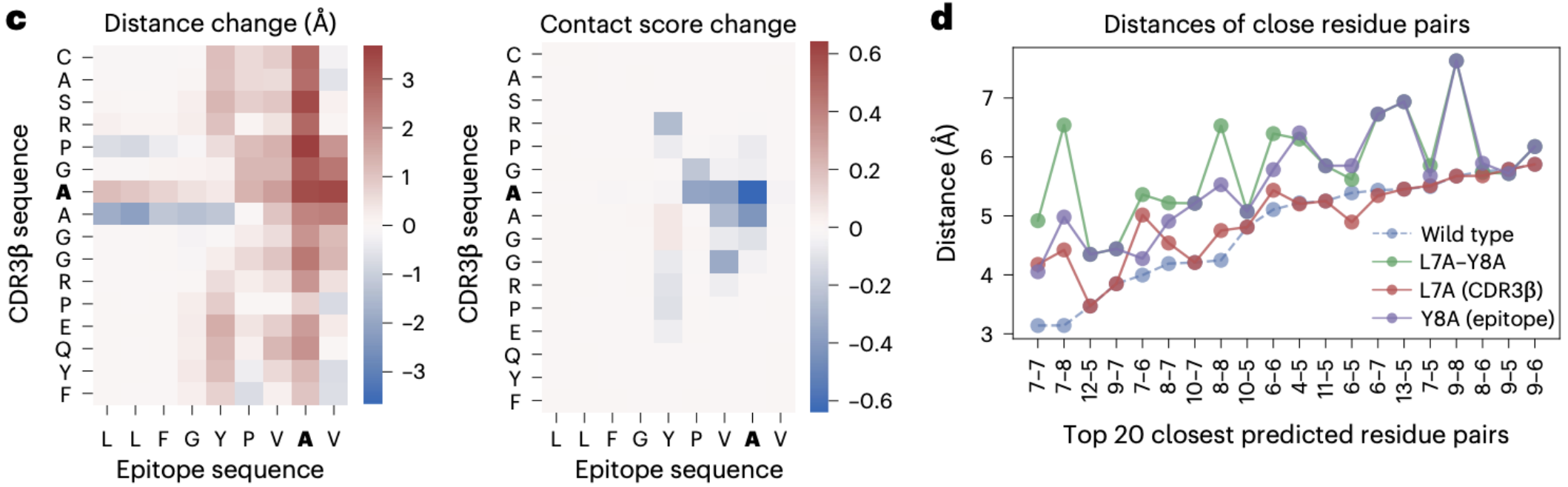
[12,20] Input

Conv1D [32] T'Conv1D

output

T cell Receptor – Epitope Binding

Nat. Machine Intel Feb 2023



Checkpoint summary

1.The reduction in the mask ratio from 0.25 to 0.15 doesn't appear to significantly aid in decreasing the peptide training loss at this stage. This may impact the performance of the two–BERT-combined model.

2. To evaluate the model's performance, we need to evaluate it using the weights of BERT trained on peptides during (1) the pre-training phase (epoch=0), (2) the initial stage (epoch=1), and (3) the phase using prot-bert pre-trained weights.

3. Consideration should be given to extending the duration of the fine-tuning stages by increasing the number of epochs.

4. Given the prevalence of methods employing BLOSUM as a dictionary or embedding, exploring this avenue may be beneficial.

5. I've encountered a roadblock where my GPU usage on the Narval cluster in Compute Canada has hit its maximum limit.

6. Currently, I'm working on reproducing other methods for comparison in our paper, testing them on various datasets.

What I tried in pan peptide learning?

1. Adjust MLM ratio (0.25 → 0.15)
2. Change max length of peptide embedding (48, 64, 128).
3. Increasing training epochs (10-->50→500)

Best result: **0.899**

Previous direct fine-tune on PROT-TRANS: **0.93**

Why?

1. Limited number of training samples
2. Not enough pre-training steps on peptide

CCDB Resources (Single GPU Mem>32G)

Nodes

Cedar

192	32	187G or 192000M	2 x Intel Silver 4216 Cascade Lake @ 2.1GHz	1 x 480G SSD	4 x NVIDIA V100 Volta (32G HBM2 memory)

Narval

159	48	498G ou 510000M	2 x AMD Milan 7413 @ 2.65 GHz 128M cache L3	1 x SSD de 3.84T	4 x NVidia A100 (mémoire 40G)

Graham (reached quota limit)

2	40	377G or 386048M	2 x Intel Xeon Gold 6248 Cascade Lake @ 2.5GHz	5.0TB NVMe SSD	8 x NVIDIA V100 Volta (32GB HBM2 memory),NVLINK

Quota Limit on Narval Cluster

Guangyi Zhang

To: Huizhong Lu via Test address <test@tech.alliancecan.ca>

Thu 6/1/2023 11:13 AM

Dear Huizhong,

I hope everything is going well. Thanks for your help and now I enjoy use Narval. However, I found most of nodes on the Narval have been unavailable since the day before yesterday. Could you help me to check if the cluster in under maintenance or if something wrong on my side? Thanks a lot!

The prompt message:

[patrick3@narval1 scratch]$ sq

JOBID USER ACCOUNT NAME ST TIME_LEFT NODES CPUS TRES_PER_N MIN_MEM NODELIST (REASON)

17422591 patrick3 def-hansenhe train_llm.sh R 17:07:45 1 10 gres:gpu:a 30G ng30708 (None)

17572162 patrick3 def-hansenhe train_prot_ber PD 1-10:00:00 1 10 gres:gpu:a 30G (ReqNodeNotAvail, UnavailableNodes:ng[10101-10104,10201-10204,10301-10304,10401-10404,10501-10504,10601-10610,10701-10712,10801-10808,10901-10906,11001-11006,11101-11106,20101-20104,20201-20204,20301-20304,20401-20404,20501-20504,30101-30104,30201-30204,30301-30302,30601-30605,30701-30712,31001-31006,31101-31104])

Cheers,
Patrick

Huizhong Lu via The Alliance Support <support@tech.alliancecan.ca>

To: Guangyi Zhang

Thu 6/1/2023 11:32 AM

Dear GuangYi,

I have increased the priority of job #17572162, it is running now.

It has not started because of lower priority:
sshare -l -A def-hansenhe_gpu
shows that the LevelFS of your group is 0.522347
(<1 means over-served)

TABLE 2

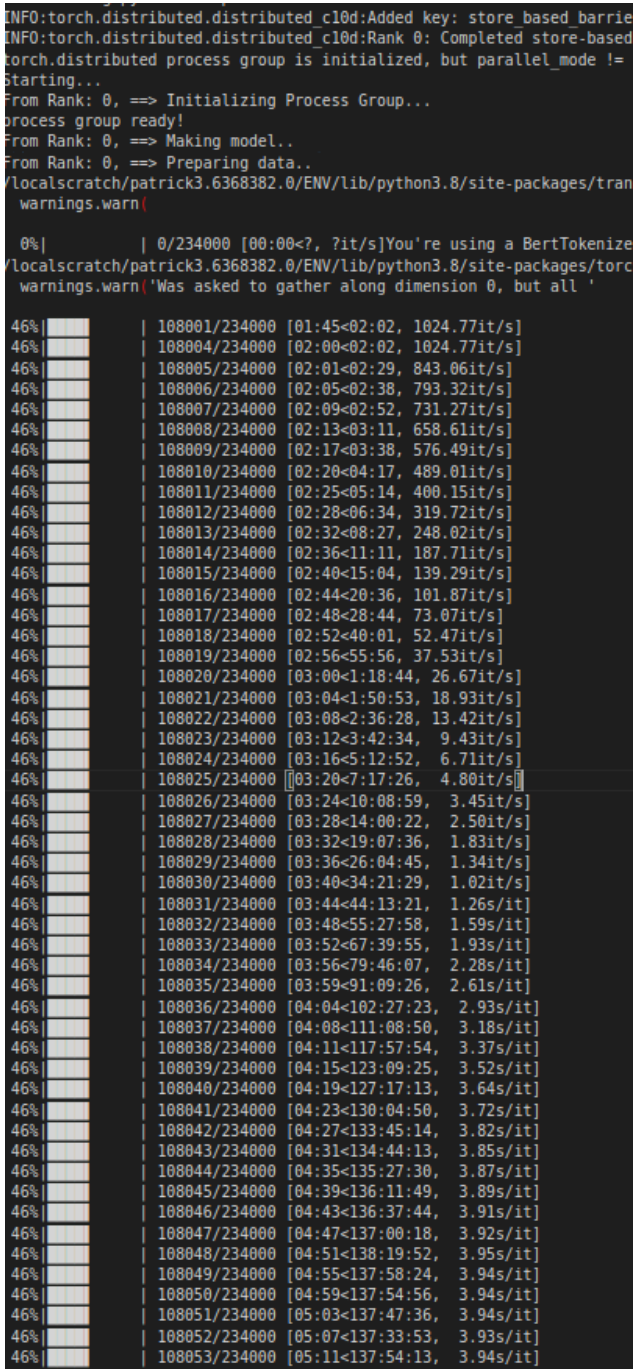
Large-Scale Deep Learning: The Table Shows the Configurations for Pre-Training the Protein LMs Introduced Here (ProtTXL, ProtBert, ProtXLNet, ProtAlbert, ProtElectra, ProtT5) Using Either Summit, a TPU Pod v2 or a TPU Pod v3

Hyperparameter	ProtTXL		ProtBert		ProtXLNet	ProtAlbert	ProtElectra	ProtT5-XL		ProtT5-XXL	
Dataset	BFD100	UniRef100	BFD100	UniRef100	UniRef100	UniRef100	UniRef100	UniRef50	BFD100	UniRef50	BFD100
Number of Layers	32	30	30		30	12	30	24		24	
Hidden Layers Size	1024		1024		1024	4096	1024	1024		1024	
Hidden Layers Intermediate Size	4096		4096		4096	16384	4096	16384		65536	
Number of Heads	14	16	16		16	64	16	32		128	
Positional Encoding Limits	-		40K		-	40K	40K	-		-	
Dropout	0.15		0.0		0.1	0.0	0.0	0.1		0.1	0.0
Target Length	512		512/2048		512	512/2048	512/1024	512		512	
Memory Length	512		-		384	-	-	-		-	
Masking Probability	-		15%		-	15%	25%	15%		15%	
Local Batch Size	8	5	32/6	30/5	2	21/2	18/7	8	4	8	4
Global Batch Size	44928	22464	32768/6144	15360/2560	1024	10752/1024	9216/3584	2048	4096	2048	4096
Optimizer	Lamb		Lamb		Adam	Lamb	Lamb	AdaFactor		AdaFactor	
Learning Rate	0.0005	0.002	0.002		0.00001	0.002	0.002	0.01		0.01	
Weight Decay	0.0	0.01	0.01		0.01	0.01	0.01	0.0		0.0	
Training Steps	40.7K	31.3K	800K/200K	300K/100K	847K	150K/150K	400K/400K	991K	1.2M	343K	920K
Warm-up Steps	13.6K	5.5K	140K/20K	40K/0K	20K	40K/5K	40K/40K	10K		10K	
Mixed Precision	FP16 Model Weight Fp32 Master Weight		None		None	None	None	None		None	
Number of Parameters	562M	409M	420M		409M	224M	420M	3B		11B	
System	Summit	Summit	TPU Pod		TPU Pod	TPU Pod	TPU Pod	TPU Pod		TPU Pod	
Number of Nodes	936		128	64	64	64	64	32	128	32	128
Number of GPUs/TPUs	5616		1024	512	512	512	512	256	1024	256	1024

Continue Training

4 GPU DDP training

CCDB has limit of 7 days training per job



Topic: Fully Transferring this project to Vivian Chu

What I can offer:

(1) All the project codes with explanations.

(2) Results of ALL my previous experiments, model checkpoints of BERT pretrained on 1M peptides.

(3) Instructions of using ComputeCanada GPUs (e.g., interactive mode, DDP job submission, continuing training) and Colab Pro+.

(4) Future directions.

(5) Transparency.

What I cannot offer:

(1) Instructions on using Vector Institute GPUs

Goals of this project

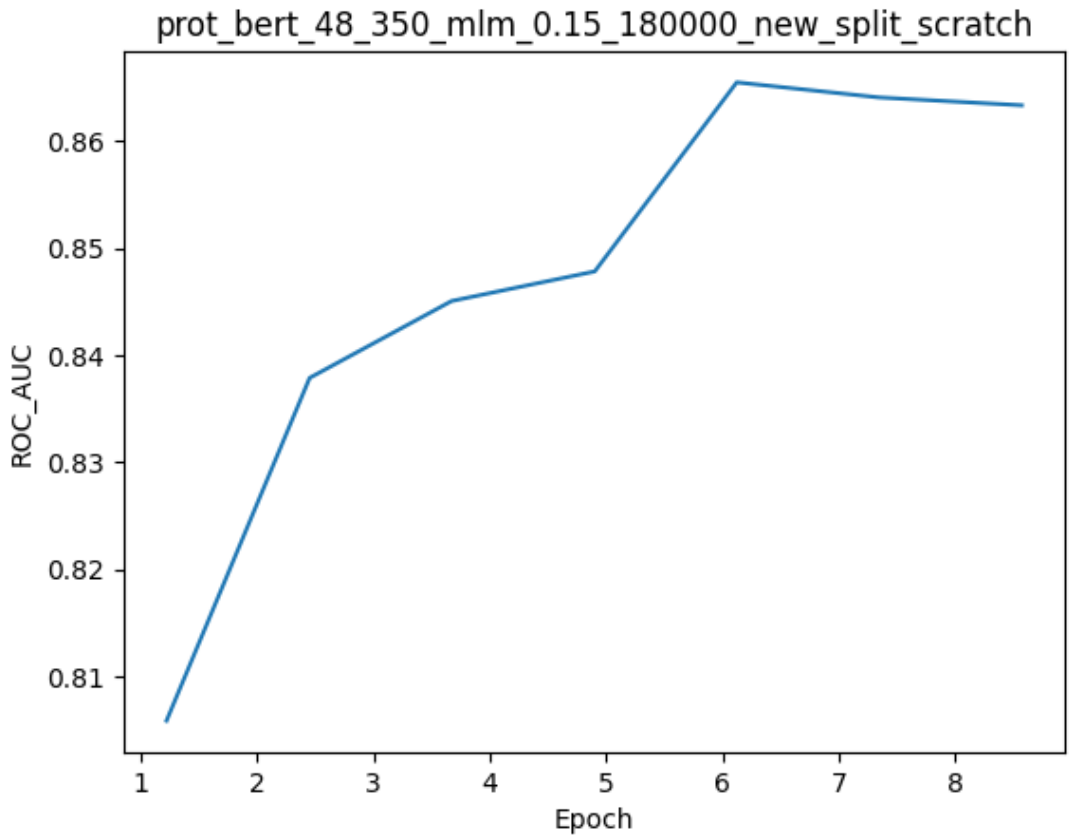
1. Novelty (Using Prot-Bert + Pre-training peptides on BERT --> Pan Peptide....)
2. Obtain state-of-the-art results across multiple public datasets
3. Validate on private and real datasets

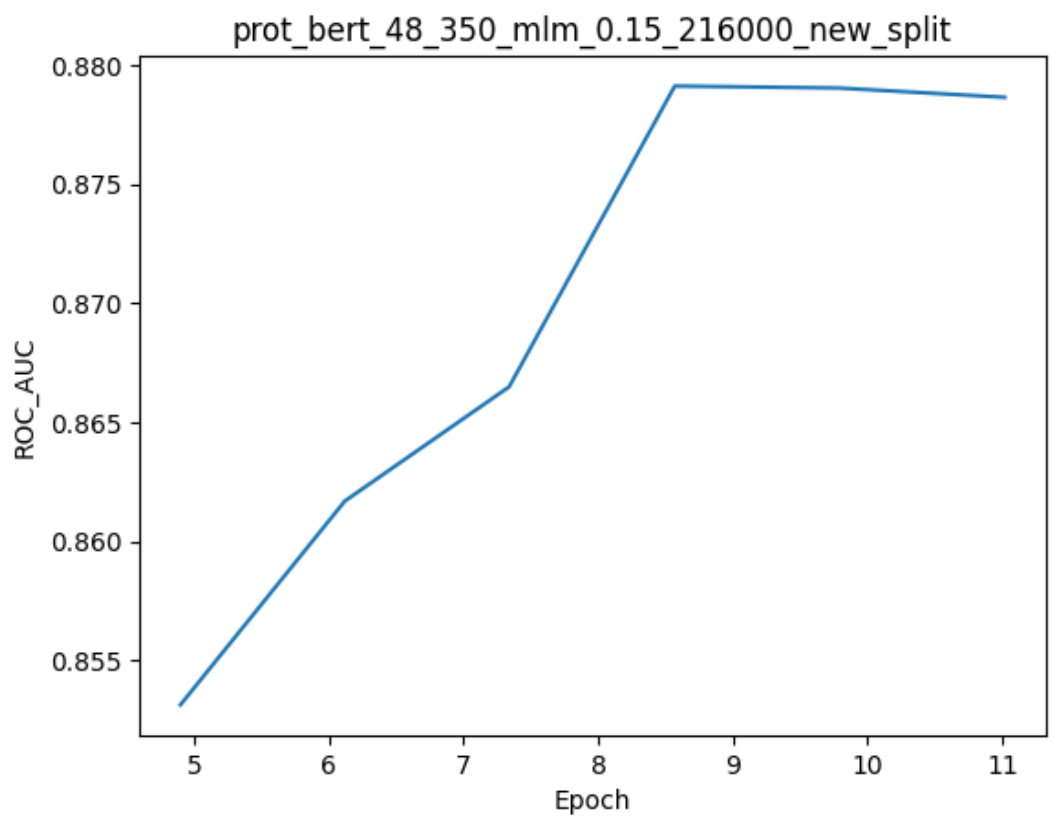
Challenges of this project

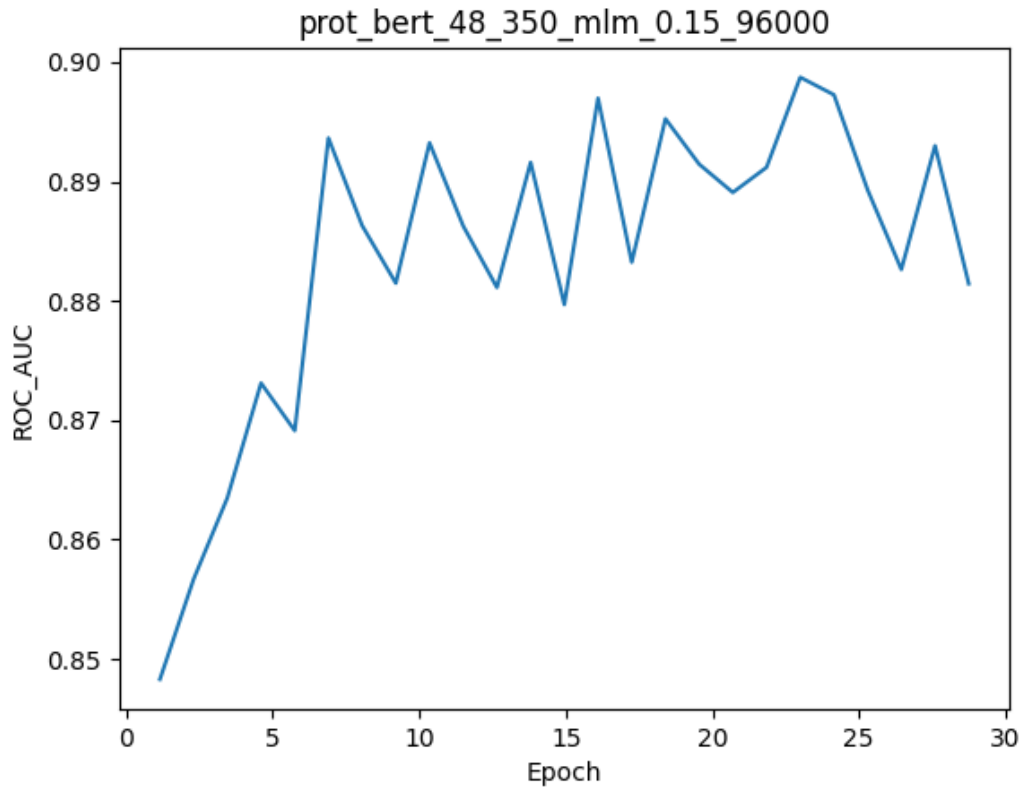
1. Very high GPU request (e.g., our method, comparison methods)
2. Very long pretraining time
3. Lack of details of LLMs pre-training on Protein (e.g., Prot-Bert). How can we figure out the suitable parameters for peptide pretraining?

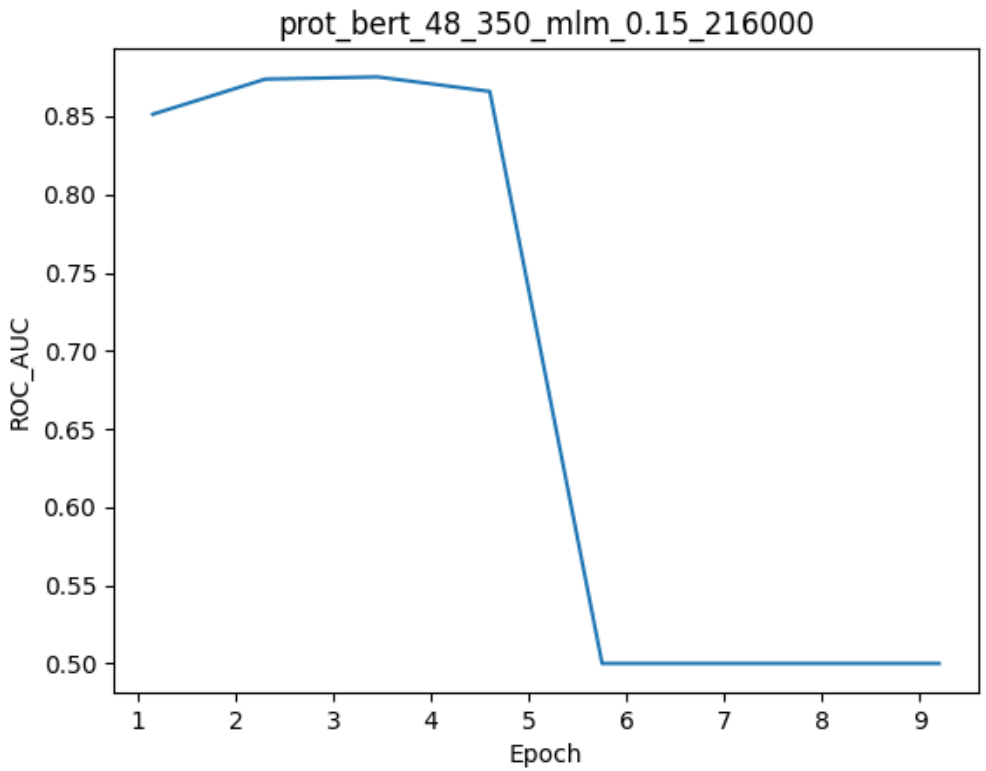
What I will focus on in the next few months (min 2, max 5)

1. Pre-training 1M peptide based on Prot-Bert weights pretrained on protein database
2. Work on MHC – split dataset only
3. Reproduced results of very recent binding papers, preparing codes. Write Introduction and related work sections.
4. Fully transfer to Vivian, as requested (keep Bo and Hansen in the loop until transfer complete).
5. Help lab mates on computing/coding upon request


prot_bert_48_350_mlm_0.15_180000_new_split_scratch
ROC_AUC
Epoch
0.81
0.82
0.83
0.84
0.85
0.86
1
2
3
4
5
6
7
8


prot_bert_48_350_mlm_0.15_216000_new_split
ROC_AUC
Epoch
0.855
0.860
0.865
0.870
0.875
0.880
5
6
7
8
9
10
11


prot_bert_48_350_mlm_0.15_96000
ROC_AUC
Epoch
0.90
0.89
0.88
0.87
0.86
0.85
0
5
10
15
20
25
30


prot_bert_48_350_mlm_0.15_216000
ROC_AUC
Epoch
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.50
1
2
3
4
5
6
7
8
9

Comparison

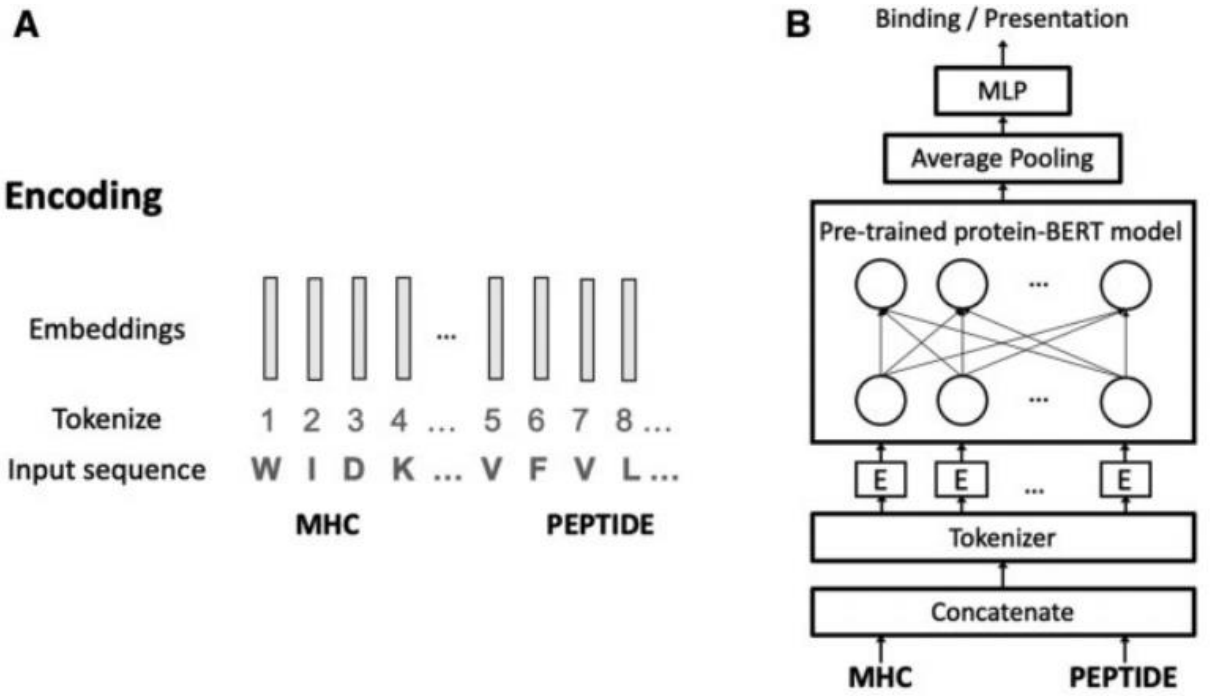
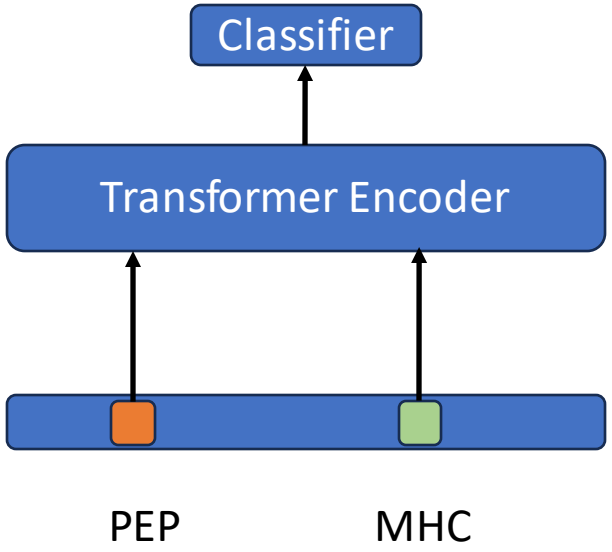


Fig. 1. (**A**) Encoding of MHC and peptide sequences. MHC and peptide sequences were first encoded into tokens with a tokenizer, where each token corresponds to a single amino acid. Peptide sequences are padded to the same length (24) with a pad token. Peptides longer than 24 are trimmed from the end. Each token is then embedded into a 768-dimensional vector with a trainable embedding layer. The tokens are integer values corresponding to the entries of the embedding matrix (parameters) of the embedding layer. (**B**) BERTMHC model architecture. The encoding layer is followed by 12 multihead self-attention layers with 12 attention heads at each layer. The outputs of the 12 heads are concatenated. Each self-attention layer transforms the input sequence into a deeper representation of the same 768 dimensions (each head transforms the input into dimension of 64). The output of the BERT architecture is then mean-pooled along the sequence dimension. A 2-layer feed forward neural network uses that to predict either binding affinity or presentation

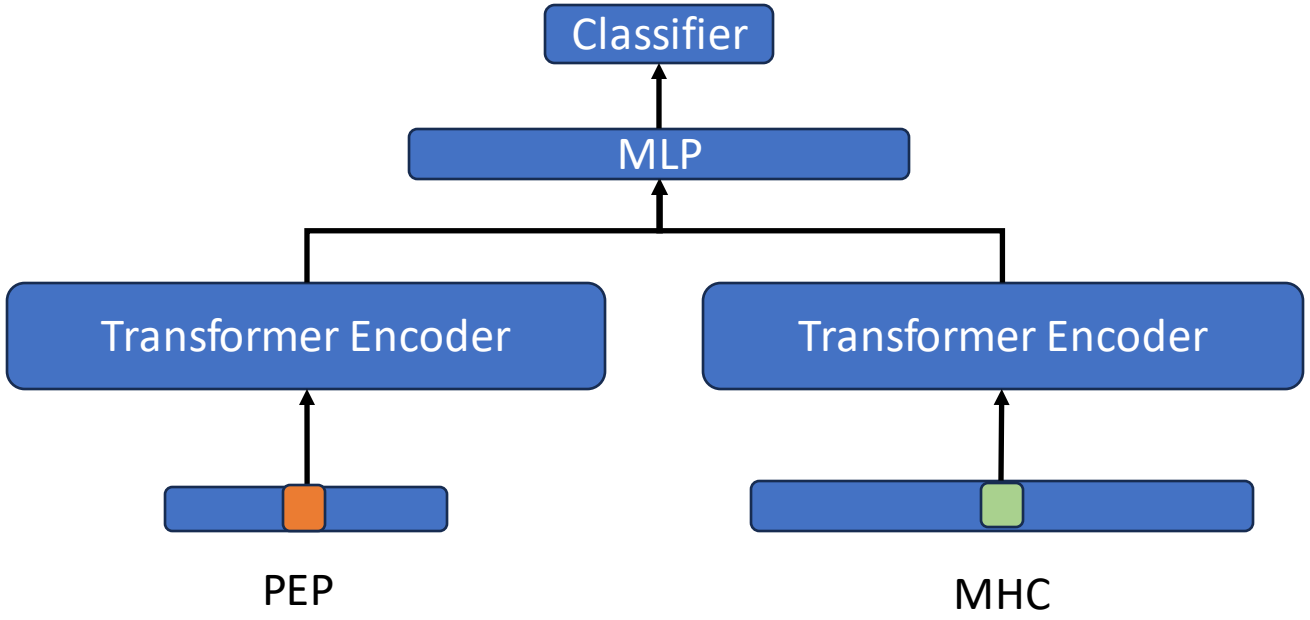
Transformer-based Networks

- One Transformer



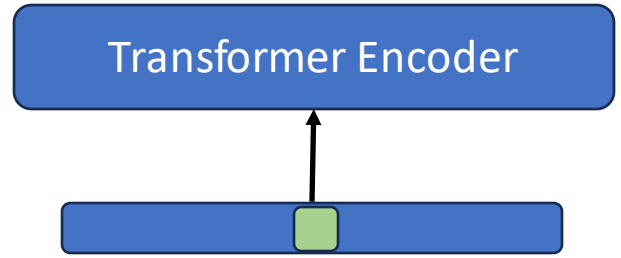
(1) Concatenation
(2) Different input ids

Simaese Transformers

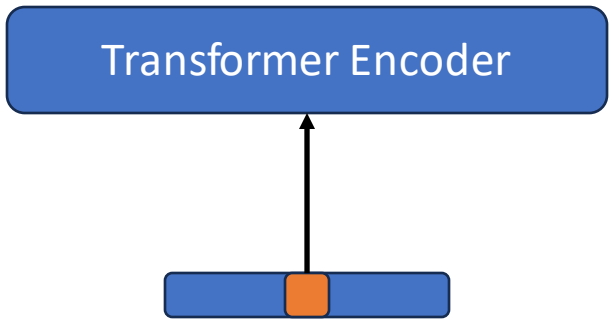


(1) Both branches using pretrained PROTBERT
(2) Pan Peptide: Training PEP branch from scratch

Siamese Transformers – Training Strategies



MHC



PEP

Pretrained weights
Protein database
(PROTBERT)

Scratch

Pre-training on
Peptide database

Pretrained weights
Protein database
(PROTBERT)

Continue pre-training
on
Peptide database

Fine-tune

Pretrained weights
Protein database
(PROTBERT)

New-MHC Split Result

Pep-MHC-I: **80-20 Split**
Unique MHC-I: **121- 30**
Samples: 392139- 98879

Model	#Trainable Params	others	AUC-ROC	AUC-PRC	PCC	PPV	F1	Sensitivity
Single PROTBERT	419M	12 hours/ epoch, total 5 epochs	0.7889	0.8833	0.5967	0.8943	0.9066	0.9193
3-layer Bi-LSTM (MHCAttNet)	0.6M	Original Paper Method (2020) 100 epochs	0.7739	0.8756	0.5813		0.9057	0.9277
Two PROTBERT (scratch-->pep pretrain)	836M	total 5 epochs	0.8654	0.9257	0.6896	0.9469	0.9160	0.8869
Two PROTBERT (scratch-->protbert weights-->pep pretrain)	836M	total 5 epochs	0.8791	0.9328	0.7217	0.9517	0.9261	0.9019
Two PROTBERT (both pep and mhc transformers using protbert weights)	836M	total 5 epochs	0.8661	0.9261	0.6902	0.9475	0.9159	0.8864